Noname manuscript No.
(will be inserted by the editor)

The State of the SBOM Tools Ecosystem and their Prevalence in OSS: A Comparative Analysis of SPDX and CycloneDX SBOM Formats

Abdul Ali Bangash · Arshdeep Singh · Tongxu Ge · Zhimin Zhao · Zitao Wang · Ankit Hans · Bram Adams

Received: date / Accepted: date

Abstract Software Bill of Materials (SBOMs) are essential components of modern software development, providing transparency and accountability by documenting components and dependencies within software projects. This paper explores the open-source tool ecosystem surrounding the two main SBOM formats, SPDX and CycloneDX, to offer insights into their usage, strengths, weaknesses, and potential areas for improvement. Firstly, the study conducts a qualitative comparison of the use cases for 108 open-source and 62 proprietary SBOM tools, identifying areas where open-source tools for each format may require enhancement. Secondly, a comparison of the health metrics for each format's tool ecosystem is made (171 CycloneDX tools vs. 470 SPDX tools). Additionally, to provide concrete insights into the challenges and potential development areas within each format, the study compares 17,795 CycloneDX and 19,195 SPDX issues from open-source tools. Finally, we investigate the characteristics of the top 250 open-source projects using CycloneDX tools and compare their health metrics with the top 250 open-source projects that use SPDX tools. The findings reveal that projects utilizing CycloneDX tools generally exhibit higher developer engagement and better overall project health compared to those using SPDX tools. This research provides valuable insights for software developers, open-source contributors, SBOM practitioners, and policymakers, informing decision-making processes around software transparency and security practices. Ultimately, it contributes to the development of a more robust SBOM tools ecosystem.

Keywords OSS, Software Ecosystem, SPDX, CycloneDx, Software Bill of Management

Abdul Ali Bangash
Software Analysis and Intelligence Lab (SAIL)
Queen's University, Kingston, ON, Canada
E-mail: abdulali.b@queensu.ca

Bram Adams
Lab on Maintenance, Construction and Intelligence of Software (MCIS)
Queen's University, Kingston, ON, Canada
E-mail: bram.adams@queensu.ca

1 Introduction

Modern software applications rely heavily on third-party dependencies, forming the foundation of their software supply chain (Cox, 2019). However, the increasing number of dependencies pose significant challenges in terms of security and reliability (Gkortzis et al., 2021). Malicious actors are known to exploit vulnerabilities in less secure dependencies, which ultimately taint the supply chain and facilitate the breach of a high-value application (Ladisa et al., 2022).

To address such vulnerabilities, it is crucial to have up-to-date information about all third-party dependencies in the supply chain of an application. In the past, the absence of proper standards for managing such information has resulted in serious consequences. Notably, in 2023, OpenAI temporarily suspended ChatGPT due to a vulnerability originating from the Redis library (Sanfilippo, 2023), exposing ChatGPT's user-information (Brudo, 2023). Similarly, the SolarWinds hack in 2020 compromised over 30,000 organizations across 190 countries, including the U.S. federal agencies, highlighting the need for robust supply chain security (Brudo, 2023). Moreover, the Log4Shell vulnerability in 2021 exploited the widely used Log4j library, necessitating prompt actions to protect systems and data (Berger, 2023).

To mitigate the risk of such software supply chain attacks, in 2021, the US government issued an executive order mandating the adoption of Software Bill of Materials (SBOMs) on companies who want to conduct business with the US (Remaley and Remaley, 2021), an endeavour followed later by others, such as the European Union, United Kingdom, Japan, and Germany (Worthington, 2023). To understand the readiness of SBOMs in the software development community, in 2022, the Linux Foundation carried out an extensive survey on 412 organizations from around the world (Hendrick, 2022). Their survey showed that more than 80% of worldwide organizations are aware of the US executive order and 76% of organizations are considering SBOM as a consequence of this executive order. Overall, 47% of organizations want scalable vulnerability reporting and 45% see SBOMs as a key method to secure the software supply chain. They also found that 52% organizations are addressing SBOMs in a few, some, or many areas of their business; 23% are addressing them across nearly all areas of their business or have standard practices that include the use of SBOMs. With the ever increasing understanding of SBOMs in general public, customers have started to ask for SBOMs from the product developers (Hendrick, 2022).

The concept of SBOM came from the notion of *bill of materials* in other engineering disciplines (van der Storm, 2007). Essentially, a SBOM is a precise specification of all "ingredients" of a given (software) product, ranging from the names and revision numbers of all source code and other files shipped with a given software release, to the names and version numbers of all third-party dependencies the product relies on, or even the licenses of those dependencies. The specific information included in an SBOM relies on the SBOM standard being used. The U.S. National Telecommunications and Information Administration (NTIA)'s general standard of SBOM outlines which data fields must be included in an SBOM file, how SBOMs should support automation, and which practices and processes should be employed when creating, distributing, and using SBOMs (Telecommunications and , NTIA). According to NTIA (Balliu et al., 2023), three existing formats meet their requirements: CycloneDX OWASP (2023a), Software Package Data Exchange

(SPDX) (SPDX, 2023c), and Software Identification (SWID) (Waltermire et al., 2016).

However, in the pursuit of employing SBOM in their supply chain, software development teams come across various problems, such as lack of generalization and lack of tool maturity (Xia et al., 2023a). In terms of generalization, neither of the two competing SBOM standards (i.e., SPDX and CycloneDX) fully meet current market needs for extensibility across software projects. Additionally, there is lack of standardization between the competing formats, as both have different properties and different purposes. SBOM practitioners believe that there should be a path of consolidation between them. In terms of maturity, they identified that current tools need to be more reliable, user-friendly, standard-conformable.

> *"The problem right now is that we are almost putting the cart before the horse – we're expecting the SBOMs to fix the problems rather than fixing the problems with an SBOM."* – A message from software development teams, security experts, and researchers. (Xia et al., 2023a)

The most fundamental and imminent concern is limited SBOM adoption by practitioners because of the lack of understanding about the SBOM tools, in terms of their technical knowledge, use cases, resource cost, and benefits.

Regardless of the standard that SBOM practitioners adopt, the SBOM creation process is fundamentally automated, relying on a variety of tools to extract key information (e.g., included source code files and dependencies) during the build process. While most information can be automatically extracted, some high-level details (e.g., file licenses) may still require human annotation or manual configuration.

However, the mere existence of an SBOM standard has no value without the availability of appropriate tools to leverage the concept effectively. For instance, tools that generate SBOMs by automatically extracting information about included source code files and dependencies are essential. These tools should be open-source and integratable into development and CI workflows to have a meaningful impact. Additionally, the sustainability and effectiveness of these tools depend on the development community. Without community support, tools cannot thrive and sustain. For example, open-source communities contribute to the continuous improvement and maintenance of these tools, ensuring they stay up-to-date with evolving standards and practices.

Once an SBOM is generated and shipped with a product release, additional tools are required to utilize the SBOM information for supply chain security. These tools might check for compliance between the SBOM specification and the release contents, ensuring that all components meet licensing and security requirements. This, in turn, requires other tools and libraries for parsing SBOM specifications across various programming languages.

Understanding and addressing the demand and issues of these tools is crucial, especially in the context of open-source development. Identifying and fixing the lacking aspects will improve support for SBOM users. The effectiveness and sustainability of SBOM tools are essential for ensuring the security and compliance of software supply chains. For example, tools that continuously monitor for vulnerabilities and compliance can prevent security breaches and legal issues, highlighting the critical need for a robust SBOM tools ecosystem.

In order to help SBOM practitioners and the research community understand and address the limitations of the current SBOM tools, in this paper, we study the tools of two major SBOM formats: SPDX and CycloneDX. We investigate the following research questions:

- RQ1: Which usecases do the current SBOM tools support?
- RQ2: What is the community-health of current OSS SBOM tools?
- RQ3: What are the common issues reported in the OSS SBOM tools?
- RQ4: What are the characteristics of Github projects that use SBOM tools?

In **RQ1**, we manually investigate which SBOM use-cases the current open-source and proprietary SBOM tools support. We find that proprietary tools are richer than OSS SBOM tools when it comes to the number of usecases they can support. Furthermore, we identify the use cases that OSS tools lack support for. Finally, we highlight the different strengths and weaknesses of each of the formats' (SPDX and CycloneDX) tools.

In **RQ2**, to investigate the overall health of open-source SPDX and CycloneDX Github tools, we employ the CHAOSS metrics (OSS health analytics metrics) (C. P. L. Foundation, Accessed May 27, 2024). We find that CycloneDX tools surpass SPDX tools in terms of contributor activity, number of commits, and issue resolution times. This information highlights the ecosystem of tools that is widely adopted and actively maintained, and also the areas that require the attention of developers.

In **RQ3**, to understanding the potential areas of improvement in the tools of each format, we extract and investigate the Github issues of the SPDX and CycloneDX SBOM tools. We manually group the most dominant issues into categories and observed their prevalence in both formats, SPDX and CycloneDX. We find that even though CycloneDX tools have more closed issues than SPDX, their issue resolution time is still faster than SPDX.

In **RQ4**, to investigate the health of projects that use SBOM tools and to understand their choice and preference in regards to an SBOM tool, we study open-source Github projects that configure the creation of SBOM files in their code or build release. We find that CycloneDX-based projects generally demonstrate better health metrics, including higher numbers of stars, watchers, releases, pull requests, and contributors. Additionally, projects have different choice of SBOM tools based on their number of commits. Projects with higher number of commits incline towards SBOM tools that provide the reporting capability, while projects with lesser commits prefer SBOM tools that provide SBOM edit support.

Our empirical study is valuable because it provides a comprehensive overview of available SBOM tools, their suitability for various use cases, and prevalent issues, enabling practitioners to make informed tooling decisions for SBOM management. It reveals that CycloneDX tools tend to have higher developer engagement, aiding practitioners in understanding the community dynamics of these formats. Furthermore, it provides valuable insights into the usage of SPDX and CycloneDX tools in real-world software projects, highlighting the critical need for a robust SBOM tools ecosystem.

2 Background and Related Work

In this section, we explain what an SBOM is, what is an SBOM specification, and which are the standard formats. We then explain how these concepts are related to our work and what kind of relevant studies have been performed in the literature.

2.1 What is an SBOM?

The concept of a bill of materials (BOM) originated from the manufacturing industry's practice of creating a document that records and helps to track and manage components used in the production of physical goods (Jiao et al., 2000). Drawing inspiration from BOM, the concept of software bill of materials (SBOM) was first formalized in 2007 by Storm et al. (2007). An SBOM is a structured document that records a detailed information about the components and dependencies comprising a software application or system (Muirí, 2019). It keeps the names, versions, and sources of third-party libraries, frameworks, modules, and other software elements used in the development process. SBOM serves as a comprehensive record of the software supply chain, enabling organizations to uniquely and unambiguously identify components of a software and their relationships, to understand and manage the security, licensing, vulnerability, and compliance aspects of their software assets.

2.2 What is an SBOM specification?

An SBOM specification refers to the detailed requirements and characteristics that define the structure, components, and behavior of an SBOM document. It outlines the standards and information that should be included in an SBOM to ensure accurate identification and tracking of software components and dependencies. The SBOM specification typically includes details on how software assets are described, organized, and documented within the bill of materials. An example of an SBOM specification is the U.S. NTIA's general standard of SBOM which outlines data fields that must be included in an SBOM file, how SBOMs should support automation, and which practices and processes should be employed when creating, distributing, and using SBOMs (Telecommunications and , NTIA).

2.3 What are the standard SBOM formats?

Standard SBOM formats follow an SBOM specification to provide a structured way to document and convey information about software components. The NTIA recognizes three main SBOM formats: SPDX, SWID, and CycloneDX (Telecommunications and , NTIA). Each format has its unique focus and strengths, catering to different stages of the software development lifecycle and varying user needs. SPDX is an open standard for SBOM that is widely used by large organizations like Intel and Microsoft due to its comprehensive nature and machine-readable consistency across industries (Linux, 2023). On the other hand, CycloneDX, an open-source standard from the OWASP community, is known for its lightweight

nature and flexibility, making it suitable for application security contexts and supply chain component analysis (OWASP, 2021). Similarly, SWID (Software Identification) tags is another format that provides a standardized way to identify and track software installations, facilitating software asset management and cybersecurity efforts (ISO/IEC, 2022). In this study, we focus on the two most prevalent SBOM formats: SPDX and CycloneDX (Das, 2023).

2.4 Related Work

Martin (2020), based on their industrial experience, discuss 11 crucial elements that an SBOM should have and also highlight nine possible usecase scenarios of an SBOM. In our work, we identify which usecase scenarios the current available SBOM tools address. However, instead of using Martin *et al.*'s taxonomy, we used the NTIA's official tool taxonomy. NTIA's taxonomy was a collective effort of multiple SBOM formats and SBOM tool vendors.

Xia et al. (2023b) addressed the limitation of secure SBOM sharing among software vendors in their work. They proposed a blockchain-based solution with verifiable credentials for secure and flexible disclosure. Their approach enhances the security, usability, and privacy of the SBOM sharing process between the software vendors. Additionally, they identified the emerging need for SBOMs in AI software (AIBOM) and highlighted the necessity of documenting co-evolving data, models, code, and configurations in an AIBOM.

Xia et al. (2023a) explored the importance of SBOMs in strengthening the security of software supply chain. Through 17 interviewees and 65 survey respondents with 17 practitioners from 15 different countries, they identified the current challenges that SBOM practitioners face. These challenges include the lack of quality in the available SBOM files, confusion between practitioners regarding its use cases, and lack of community help. In our work, we address these challenges by identifying the usecases SBOM tools support and shedding light on the community traits of these tools.

Bi et al. (2023) perform a comprehensive survey on 4,786 GitHub discussions of projects that use SBOMs.Their work highlights key topics, challenges, and solutions intrinsic to the effective utilization of SBOMs. However, in this paper, we focus on the topics and challenges of SBOM tools instead of challenges related to SBOM usage in general.

Nocera et al. (2023)'s work is closely related to our work's RQ4, in which they examine the usage of SBOM tools in 186 Github projects. However, unlike us, they focus on studying the actual SBOM files available in these projects and how frequently they are updated, rather than analyzing the traits of the projects that employ SBOM tools. Additionally, they do not differentiate between the two formats, SPDX and CycloneDX, when deriving their results, and their findings are demographic in nature.

Mirakhorli et al. (2024)'s work is closely related to our work's RQ1, in which they evaluate 84 open-source and proprietary SBOM tools. To identify their emerging use cases, they execute 5 SBOM tools and demonstrate their interoperability issues. Our study, on the other hand, identifies the usecases of 187 SBOM tools using the NTIA's official taxonomy. Furthermore, instead of carrying out a qualitative investigation of 5 tools, we quantitatively analyze the issues of 641 SBOM

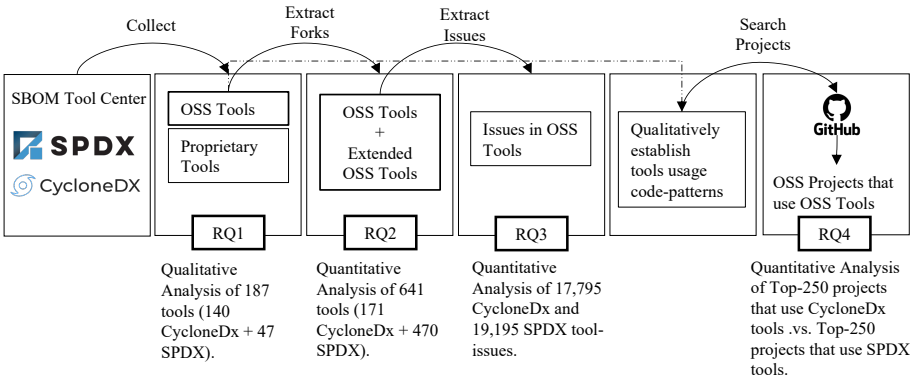


Fig. 1: Overview of our methodology and dataset.

tools to highlight the most prevalent problems within them. Lastly, our study extends beyond the investigation of use cases and problems of SBOM tools, as it further looks into the health factors of the projects that use these tools.

The CHAOSS metrics that we use in RQ3 and RQ4, in order to calculate the overall health of open-source software, have been previously used to study the process and impact of onboarding new contributors in open-source software ecosystems, using an observational study and empirical validation to analyze teaching strategies, challenges, benefits, and the effects on diversity, productivity, and code quality (Foundjem et al., 2021).

3 Overview of the Study

We have provided a general overview of our methodology in Figure 1. First, we collect all the SBOM tools available at the tool center of both formats (SPDX and CycloneDX), we then qualitatively analyze these tools to identify which use-cases they support and how they are different from one another (RQ1). Second, we extend the initially collected open-source tools dataset with the open-source tools that are available on Github to compare both of the formats (SPDX and CycloneDX) quantitatively (RQ2). Third, we extract the Github issues of all the collected open-source tools and quantitatively analyze the most prevalent type of issues (RQ3). Finally, we qualitatively curate a list of search-patterns from the previously collected list of open-source tools to identify projects on Github that use these tools, after collection, we study the characteristics of these projects to conclude which type of projects prefer which one out of the two formats.

3.1 Collecting the SBOM tools (RQ1)

To investigate the different use cases of CycloneDX and SPDX tools, on 17th of January 2024, we extracted all the tools available on the official tool center websites of CycloneDX (OWASP, 2023b) and SPDX (SPDX, 2023a,b).

We found 219 tools on the CycloneDX tool center website and 47 tools on the SPDX tool center website. On closer inspection, we found some discrepancies in the information provided on the websites of both the tool centers i.e., we found that not all the tools mentioned as open-source by their developers are necessarily open-source, such tools have most of their functionalities private and commercial. Secondly, we discovered that some of the tools in the CycloneDX tool center also support the SPDX format, and similarly, certain tools in the SPDX tool center support the CycloneDX format as well. We also identified certain SPDX tools whose mentioned use cases on SPDX tool center do not necessarily provide the complete list of use cases that it supports. We also identified certain SPDX tools whose use cases, as listed on the SPDX tool center, may not reflect the complete range of use cases the tool supports. To determine the complete range of the usecases of these tools, and to determine which ones are truly open-source, we manually analyze them.

3.2 Manual Analysis to identify the usecases (RQ1)

With a 95% confidence interval and an error margin of 5%, we randomly selected 140 tools from the 219 tools that were available on CycloneDX tool center, and selected all 47 tools that were available on the SPDX tool center. This step leaves us with 187 tools to analyze in total. Two of the authors agreed to manually investigate the tools to categorize them as open-source/proprietary and if they support a single format or both (SPDX and CycloneDX). In addition, to highlight the use cases that a tool supports, the authors categorized these tools according to the official NTIA Tool Classification Taxonomy (Telecommunications and , NTIA). This tool classification taxonomy was created by NTIA after surveying several owners and SBOM tool developers to provide a variety of SBOM management use cases related to SBOM (NTIA, 2021a).

- The *Produce* category includes tools that automatically create SBOMs during software artifact creation, analyze source or binary files to generate SBOMs, and facilitate manual data entry.
  - **Build** - Tools, such as Valaa Stack[1], that automatically generate an SBOM as part of the software build process, containing information about the build.
  - **Analyze** - Tools, such as SecObserve[2], that analyze source or binary files to generate an SBOM by inspecting artifacts and associated sources.
  - **Edit** - Tools, such as CycloneDX PHP Library[3], that assist in manually entering or editing SBOM data.
- The *Consume* category includes tools that allow users to view SBOM contents in a human-readable form, compare multiple SBOMs to identify differences, and import SBOMs for further processing and analysis.

[1] `https://github.com/valaatech/kernel/blob/8e24e9e731bb67d0da82fbd56b6a8e32f22c178f/CHANGELOG.md?plain=1#L841`

[2] `https://github.com/MaibornWolff/SecObserve/blob/27db653939e6209f8b6a24d3a45a76c08f358086/docs/usage/supported_scanners.md`

[3] `https://github.com/CycloneDX/cyclonedx-php-library`

  - **View** - Tools, such as Tidelift[4], that display SBOM contents in a human-readable form, supporting decision-making and business processes.
  - **Diff** - Tools, such as SBOMTrend[5], that compare multiple SBOMs and highlight differences, such as between different versions of software.
  - **Import** - Tools, such as NetRise Turbine[6], that enable the discovery, retrieval, and import of SBOMs into systems for further processing and analysis.
- The *Transform* category includes tools that translate SBOMs between file types, merge multiple SBOM sources for analysis and audit, and support integration with other tools through APIs and object models.
  - **Translate** - Tools, such as dbom[7], that convert an SBOM from one file type to another while maintaining the same information.
  - **Merge** - Tools, such as SBOM Assembler[8], that combine multiple SBOM sources and other data for comprehensive analysis and auditing.
  - **Tool Support** - Tools, such as Enso[9], that facilitate SBOM use in other applications through APIs, object models, libraries, and other reference sources.

To avoid bias regarding the manual classification, as a first step, two authors randomly chose and categorized 29 out of 187 tools. During this round, the authors noted different sources they had to look at to categorize the tools. The inter-rater reliability agreement for the tool categorization between the two authors yielded a Cohen's Kappa of 0.72, this value indicates "substantial agreement" between the raters Gisev et al. (2013). After the first round, the authors discussed each one of their approaches to categorize the tools. The authors resolved conflicts and agreed upon a streamlined process to categorize the rest of the tools. The authors agreed upon a unified search approach to continue their manual analysis on the rest of the tools.

After agreeing upon the categorization criteria and results of the 29 tools, our manual analysis for the rest of the 158 tools entails reading the tool description available on the CycloneDX or SPDX tool center website and some additional exploration based on the types of tools: open-source or proprietary. In regards to open-source tools, we read through the Readme of their GitHub repository. In case the Readme did not provide sufficient information, we searched the GitHub repository with the following keywords separately: "sbom", "spdx", and "cycloneDx", we then read the relevant Markdown or Text format files if available. If there is no such file available, then we look at the usage of "spdx" and "cycloneDx" keywords in the code of these repositories. The code usually provides us with an idea of a tool if it is capable of generating, consuming, or transforming an SBOM. The code also highlights if a tool can support both formats or just one.

In regards to proprietary tools, we search for the keywords "sbom", "spdx", and "cycloneDx" initially on the landing web-page of the tool. This would sometimes

[4] https://support.tidelift.com/hc/en-us/articles/4406286154004-About-projects-and-bill-of-materials#obtaining-a-bill-of-materials-0-1

[5] https://github.com/anthonyharrison/sbomtrend

[6] https://www.netrise.io/platform/software-bill-materials-management

[7] https://github.com/HaRo87/mdbom

[8] https://github.com/interlynk-io/sbomasm

[9] https://knowledge.enso.security/connectors-and-integrations/connectors-and-marketplace

provide us with information on the use cases and format support type of the tool. If we are not able to find the relevant information, we search the same keywords along with the tool name on the Google search engine. This search would yield our information from a variety of sources spanning from LinkedIn or Medium blogposts written by the tool developers to the tool's documentation and demo tutorials. In case we find a tool demo or documentation, we read through it to understand its use cases, while in the other case, when there is only a blog post available, we read through the blog post to understand what the tool does.

Both formats' tool-centers support tags along with tool names as well, while these helped us identify the primary usecase of the tool, on a closer inspection, we found that the tags do not sufficiently provide the complete picture of which usecases the tool supports. Both of the tool centers also had some duplicate tools but they were difficult to identify due to different naming conventions, therefore, we mapped the official urls of the tools and identified 15 duplicates, with 1 exception in which one url carried two different tools.

In this final round of manual analysis, the inter-rater reliability agreement yielded a Cohen's Kappa of 0.85, this value indicates "nearly perfect agreement" Gisev et al. (2013). To finalize the classification of the tools, the authors discussed and resolved their conflicts. In Table 10, we present the results of 170 distinct tools, out of which 62 are proprietary and 108 are open-source tools. This table lists open-source tools first and proprietary tools later, all tools sorted by their names in alphabetical order.

3.3 Extending the collection of OSS SBOM tools (RQ2)

To collect open-source SBOM tools that support SPDX and CycloneDX, we extract the forks of the open-source tool Github repositories that were available on SPDX and CycloneDX tool-center. One of the authors manually analyzed the forked repositories and filtered out forks that did not expand over the parent tool i.e., have no further commits after the fork, and also filtered out the forks that did not have a description in the Readme file from which we can understand if it is a tool repository or not.

As a result, we found 171 CycloneDX repositories and 470 SPDX repositories that represent a tool project. Even though, when compared to SPDX tool-center, CycloneDX tool-center provides a larger number of open-source tools, our Github search reveals more SPDX tools (470) than CycloneDX tools (171). A larger ecosystem of tools could suggest a wider variety of functionalities and potential benefits for users in managing software bill of materials (SBOMs) effectively.

3.4 Extracting the issues and their tags of the OSS SBOM tools (RQ3)

We queried the 171 CycloneDX and 470 SPDX tool Github repositories to collect their issues. Since not all the repositories contain issues, we identified 17,795 issues in 90 CycloneDX tool repositories and 19,195 issues in 191 SPDX tool repositories. Since CycloneDX's first prototype was released in May 2017, we filter out all the issues that were created before 2018. After this filtration step, we were left with 17,093 issues in 90 CycloneDX tools and 16,747 issues in 171 SPDX tools.

For analyzing the types of issues, we consider only those issues that are already tagged by tool maintainers on Github. This step leaves us with 19,812 total tags associated to 16,747 issues of 171 SPDX tools (with 361 unique tags), and 22,302 total tags in 17,039 issues associated to 90 CycloneDX tools (with 360 unique tags).

We sort the tags by their number of occurrence and investigate their distribution. To investigate the most prevalent tags only, we consider the fourth quartile of this distribution. For CycloneDX, we find 92 unique tags that are associated with 27 to 5815 issues. For SPDX, we found the 91 unique tags that are associated to 28 to 3979 issues. The first author then manually investigated these 92 CycloneDX and 91 SPDX tags to exclude the ones that do not convey the meaning of an issue by their name, such as "P1", "minor", "cannot reproduce", "super", "easy", and "not an issue", etc. This filtration leaves us with 49 most prevalent unique tags in CycloneDX tools and 48 most prevalent unique tags in SPDX tools. Furthermore, tags with the same meaning were merged, such as "technical debt" and "technical-debt", "license scan" and "license", and "new feature" and "nice to have" etc. The complete list of tags and their categories is available on our replication package[10].

After merging the tags, we manually group the similar tags in a broader category. For this process, the first author sorts the tags and groups the ones that have similar purposes, such as "defect", "technical issue" and "bug", categorized as "Bug fixes and Defects", docs and documentation categorized as "Documentation", and "enhancement", "must have", "nice to have" and "feature", categorized as "Feature Development and Enhancement". The last author then, having the most experience, re-analyzed the categories of the tags and raised 10 concerns about the grouping scheme. These concerns were related to: the possible duplication of tags, such as CycloneDX having two tags named "stale"; mis-judgement of categories e.g., "docs" falling into "CI/CD" category instead of "Documentation"; and better naming convention, e.g., "Bug Fix" should be "Bug Fixes and Defects". Through a collaborative process, the authors refined and finalized 14 distinct categories consisting of 97 tags (CycloneDX: 49, SPDX: 48), as detailed in Section 4.3.

3.5 Collecting OSS projects that use OSS SBOM tools (RQ4)

In order to collect projects that use SBOM tools, we could have selected projects with an SBOM file in their repository. However, projects typically do not release their SBOM files in the repository because these files are generated as an outcome of the build process within their build systems, such as Maven Central carries SBOM files for Java projects. Since we wanted to determine which tools are being used to generate SBOMs, we searched for traces of SBOM tool initialization in the repositories of GitHub projects.

Out of the open-source SBOM tools that are available on the official tool-center website of SPDX and CycloneDX, and also support the SBOM creation (Build) usecase from the NTIA's tool taxonomy, we analyze their codebase and the usage guidelines in their Readme file to identify the source-code command they require to initiate an SBOM file in a project. We call such commands *creation-patterns*. We

[10] https://github.com/SAILResearch/wip-24-ali-sbom-emse

Table 1: SPDX and CycloneDX Tools and corresponding pattern used to find repositories using these tools.

Tool Name	Code Pattern for SBOM creation
bom	`kubernetes-sigs/bom, bom generate`
ort	`oss-review-toolkit/ort`
protobom	`bom-squad/protobom/pkg/sbom, bom-squad/protobom`
sbom-tool	`microsoft/sbom-tool/releases, sbom-tool generate`
spdx-maven-plugin	`<artifactId>spdx-maven-plugin</artifactId>` `createSPDX`
spdx-sbomgenerator	`opensbom-generator/spdx-sbom-generator`
Spdx-Java-Library	`<artifactId>java-spdx-library</artifactId>` `Spdx-Java-Library`
tools-golang	`spdx/tools-golang`
tools-java	`tools-java-1.1.5-jar-with-dependencies.jar,` `spdx/tools-java`
tools-python	`pyspdxtools -i, spdx_tools.spdx.model,` `pypi/spdx-tools`
cyclonedx-cli	`CycloneDX/cyclonedx-cli`
cyclonedx-conan	`CycloneDX/cyclonedx-conan`
cyclonedx-core-java	`<artifactId>cyclonedx-core-java</artifactId>`
cyclonedx-dotnet	`cyclonedx-dotnet-generate, cyclonedx-dotnet`
cyclonedx-dotnet-library	`CycloneDX.Core, CycloneDX.Utils, CycloneDX.Spdx,` `CycloneDX.Spdx.Interop`
cyclonedx-gomod	`cyclonedx-gomod, CycloneDX/cyclonedx-gomod`
cyclonedx-go	`cyclonedx-go`
cyclonedx-javascript-library	`require('@cyclonedx/cyclonedx-library'),` `.Enums.ComponentType.Application`
cyclonedx-maven-plugin	`CycloneDX/cyclonedx-maven-plugin,` `cyclonedx-maven-plugin`
cyclonedx-node-module	`npx cyclonedx-bom, cyclonedx-node-module`
cyclonedx-node-npm	`cyclonedx-npm`
cyclonedx-node-pnpm	`cyclonedx-node-pnpm`
cyclonedx-php-composer	`cyclonedx/cyclonedx-php-composer`
cyclonedx-php-library	`CycloneDX/cyclonedx-php-library`
cyclonedx-python-lib	`CycloneDX/cyclonedx-python-lib`
cyclonedx-ruby-gem	`cyclonedx-ruby-gem, cyclonedx-ruby`
cyclonedx-rust-cargo	`CycloneDX/cyclonedx-rust-cargo`
cyclonedx-webpack-plugin	`new CycloneDxWebpackPlugin`
cdxgen	`cdxgen -t, .verify(bomSignature),` `npm:@cyclonedx/cdxgen`

identified 66 creation-patterns for these open-source SBOM tools (16 for SPDX tools and 55 for CycloneDX tools). All of the creation-patterns are available in our replication package, however, a few of them are displayed along-with-their tool names in Table 1. To identify projects that use these SBOM tools in order to create SBOM files, we then use the Github search API to identify projects that carry these creation-patterns in their code files or build configuration files.

The search queries for the projects that use SPDX tools or CycloneDX tools to create SBOMs returned $3,944$ repositories. After removing duplicates and the source repositories of the tools themselves, we ended up with $2,426$ repositories in total, comprising 307 projects that use SPDX tools, 1087 projects that use CycloneDX tools, and the remaining projects use both SPDX and CycloneDX

Table 2: Manual classification of 170 SBOM tools (19 exclusively support SPDX, 73 exclusively support CycloneDX, while 78 support both formats) in NTIA's tool taxonomy.

Category	Type	Description	SPDX	CycloneDX
Produce	Build	SBOM creation	74.36%	79.35%
	Analyze	Artifact analysis	70.51%	57.61%
	Edit	Manual SBOM editing	8.97%	6.52%
		Total	94.87%	92.39%
Consume	View	Visualization of decision making	67.95%	46.74%
	Diff	SBOM comparison	10.26%	6.52%
	Import	SBOM retrieval	43.59%	23.91%
		Total	79.49%	55.43%
Transform	Translate	File type conversion	14.10%	15.22%
	Merge	SBOM aggregation	11.54%	7.61%
	Tool Support	Integration support	37.18%	52.17%
		Total	46.15%	63.04%

tools. In our analysis, to compare SPDX and CycloneDX, we consider projects that use only one of the two formats (1,394). Of these 1,394 projects, the first set of 307 projects uses SPDX tools, while the second set of 1087 projects uses CycloneDX tools. We then sort the projects by their number of GitHub stars and select Top-250 projects from each set. Selecting projects based on the number of GitHub stars ensures a focus on popular and well-supported projects, reflecting community interest and active development (Borges and Valente, 2018).

To summarize, we compare two sets, the first set representing Top-250 projects (by stars) that use CycloneDX tools, and the second set representing Top-250 projects (by stars) that use SPDX tools.

4 Results

4.1 Usecases of the SBOM Tools Ecosystem (RQ1)

In RQ1, we study the usecases of the open-source SBOM tools that support the CycloneDX and SPDX formats. Identifying different use cases would allow the SBOM practitioners to compare the tools and adopt or modify the most suitable ones for their needs. Such an identification process will also highlight the capabilities and limitations of the available tools, which will help SBOM tool makers improve their tools.

4.1.1 SPDX.vs. CycloneDX Tools.

Proprietary tools have a higher support for dual-format (i.e., supporting SPDX and CycloneDX at the same time) than open-source tools. Across the 170 tools that we manually analyzed, 78 tools support both SPDX and CycloneDX even if they are mentioned on the tool-center webpage of only one

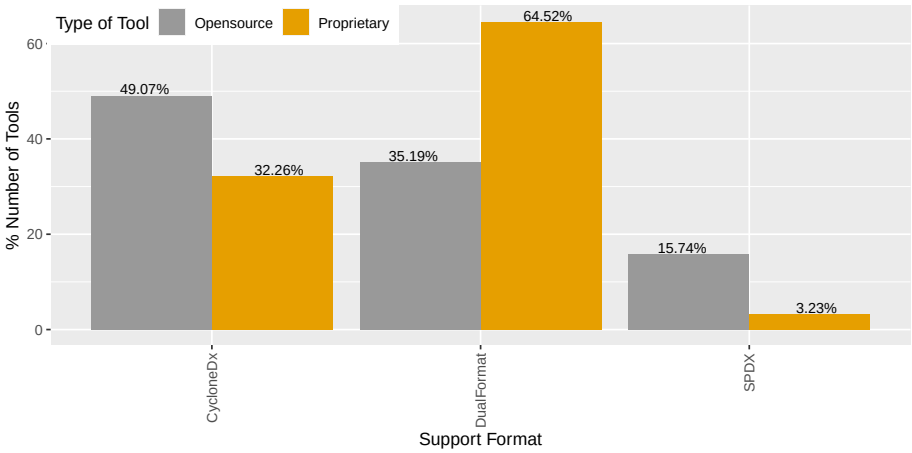


Fig. 2: Distribution of 170 manually analyzed SBOM tools across the SBOM specifications that they support.

of the formats. Eventually, for comparison, we group the tools into three categories: SPDX tools that support SPDX only (19), CycloneDX tools that support CycloneDX only (73), and dual-format tools that support both formats (78). We visualize these results in Figure 2.

This figure shows that tools that explicitly support CycloneDX and SPDX mostly adopt the open-source format, while dual-format tools are mostly proprietary. In other words, proprietary tools are more inclined towards supporting both formats (64.52%) while open-source tools are more inclined towards supporting a single format (64.81%).

RQ1: Open-source .vs. Proprietary in terms of format support.

Proprietary tools are more inclined towards supporting both formats (64.52%), while open-source community tools are more inclined towards supporting a single format at a time (64.81%). Despite open-source community tools primarily focusing on supporting a single format, there is an opportunity to enhance interoperability by aligning with the trend of proprietary tools that support multiple formats.

CycloneDX tools lack support for the *Diff* usecase, dual-format tools lack support for the *Support* usecase, and SPDX tools lack support for the *Build* usecase. Furthermore, comparatively, CycloneDX tools have a greater support for the *Build* and *Support* usecase, dual-format tools have a greater support for the *Import* usecase, while SPDX tools have a greater support for the *Diff* and *Translate* usecase. We now look into the usecases that each of the tool-category (CycloneDX, SPDX, dual-format) support. The results of this analysis are represented in Figure 3. Although the figure shows that SPDX tools are behind in every usecase, the interpretation of SPDX lagging

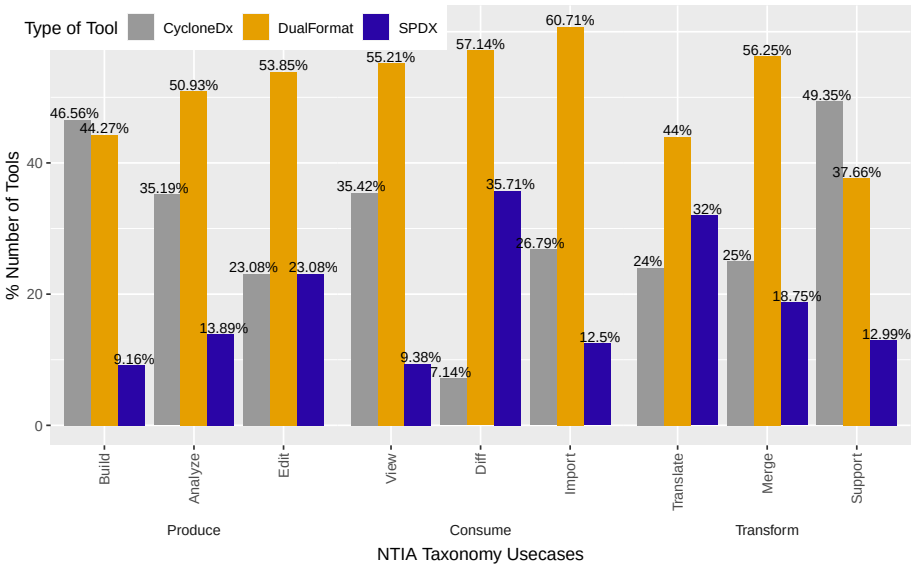


Fig. 3: SBOM specification format distribution of 170 manually analyzed tools across Open-source .vs. Proprietary.

behind the other formats might not be entirely correct especially provided the fact that there are only 19 tools that explicitly support SPDX, as compared to 73 for CycloneDX and 78 for dual-format support. For a fair comparison between the distributions, we consort towards the chi-square test of random distribution. The chi-square test found a significant difference between the tools by their support format across the usecases that they support, with < 0.05 significance level, and a "Strong" Cramer's effect size of 0.181 (Kampenes et al., 2007). Since the tools by their support type are significantly different in their coverage of usecases, we further investigate the Cramer's effect size to identify the usecases that have a strong to weak or negative to a positive association with a certain support format. To investigate the associations, we calculate Pearson residuals of each observation from the chi-square test. In this case, the Pearson residual highlights the strength of the association between a usecase to the tool type by its support format. We present the residual values in Table 3, and visual representation of the same data, for an easier interpretation, in Figure 4. The figure shows that CycloneDX tools are negatively associated ("not associated") with the *Diff* usecase. Dual-format tools are negatively associated with *Support* usecase. Furthermore, SPDX tools have a lack of support for the *Build* usecase. On the other hand, CycloneDX tools are positively associated with the *Build* and *Support* usecases, dual-format tools are positively associated with the *Import* usecase, and there is an attraction between SPDX tools and the *Diff* and *Translate* usecase. The size of the circle in the figure represents the contribution of a tool type towards a usecase.

We now use the residual values to investigate which usecases have contributed the most towards the chi-square value (the difference between the tools based on the format they support), this result will provide us with an insight into the

Table 3: The Pearson residuals and contribution proportion (%) comparison between the usecases of SPDX, CycloneDX, and dual-format (DF) SBOM tools.

Cateogry	Usecase	SPDX	CycloneDX	DF	SPDX (%)	CycloneDX (%)	DF (%)
Produce	Build	-1.334	1.733	-0.812	5.054	8.530	1.872
	Analyze	0.129	-0.362	0.248	0.047	0.372	0.174
	Edit	0.949	-0.84	0.236	2.555	2.005	0.158
Consume	View	-1.085	-0.304	0.831	3.341	0.263	1.962
	Diff	2.275	-1.848	0.421	14.688	9.695	0.502
	Import	-0.19	-1.29	1.222	0.103	4.722	4.239
Transform	Translate	2.533	-1.09	-0.374	18.213	3.371	0.398
	Merge	0.58	-0.806	0.399	0.956	1.846	0.451
	Support	-0.107	1.729	-1.449	0.032	8.488	5.963

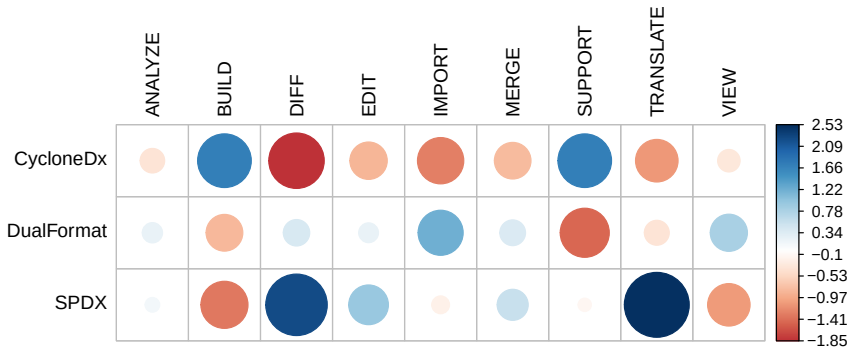


Fig. 4: Visualization of the Pearson residuals to compare the usecases of SPDX, CycloneDX, and dual-format SBOM tools.

usecases whose frequency is mainly different between the SPDX, CycloneDX, and dual-format tools. The residual values contribution (%) is presented in Table 3. From this table, it can be observed that the most contributing tool-type/usecase residuals to the chi-square are *Translate* (21.58%), *Diff* (24.38%), and *Build* (13.58%). These three usecases contribute 59.54% to the total chi-square score and thus account for most of the difference between SPDX, CycloneDX, and dual-format tools.

RQ1: Usecases support of SPDX.vs. CycloneDX.

We find that tools that explicitly support SPDX mostly support the *Translate* usecase and lack the *Build* usecase. On the other hand, we find that tools that explicitly support CycloneDX mostly support the *Build* usecase, while lack the *Diff* usecase. These results help SBOM tool-developers understand the distinct strengths and weaknesses of SPDX and CycloneDX formats, guiding them in tailoring their tools to better cater to specific use cases.

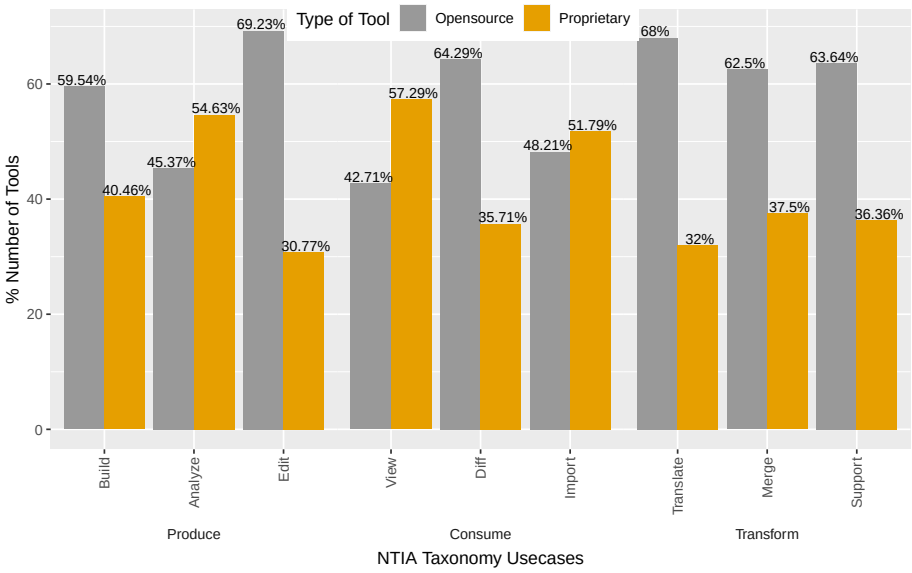


Fig. 5: Comparison of the distribution of 170 SBOM tools across NTIA tool taxonomy with the distinction of open-source and proprietary.

4.1.2 Open-source vs Proprietary SBOM Tools.

Proprietary SBOM tools offer more use cases than the open-source SBOM tools. We manually identify the NTIA's taxonomy usecases that 108 open-source tools and 62 proprietary tools support and compare them to find out if the open-source tools are as resourceful as the proprietary ones. Initially, we ran the Chi-square test of independence and found a statistically significant difference between the open-source tools and the proprietary tools, with < 0.05 significance level, and a "Strong" Cramer's effect size of 0.172 (Kampenes et al., 2007).

RQ1: Proprietary tools cover more usecases.

In general, despite being fewer in number compared to the open-source tools, proprietary SBOM tools provide 1.14 times more use cases than open-source SBOM tools. Our results imply the OSS community to focus on usecases in which they are lagging behind the Proprietary tools, such as *Build*, *Analyze*, *View*, and *Support*.

Proprietary tools moderately support the *Analyze* and *View* usecases, while the open-source tools lack support for these usecases. On the other hand, open-source tools have a greater availability for the *Support* usecase, while the proprietary tools have a lack of availability for this usecase. Since the proprietary and open-source tools are significantly different in their coverage of use-cases, we further investigate the Cramer's effect size to investigate which type of tools have a strong to weak, or negative to positive

Table 4: The Pearson residuals and contribution proportion (%) comparison between the usecases of Open-source .vs. Proprietary SBOM tools.

Cateogry	Usecase	Proprietary	Open-source	Proprietary (%)	Open-source (%)
Produce	Build	-1.052	0.986	6.027	5.29
	Analyze	1.199	-1.123	7.825	6.868
	Edit	-0.842	0.789	3.864	3.391
Consume	View	1.512	-1.416	12.444	10.922
	Diff	-0.604	0.565	1.983	1.741
	Import	0.552	-0.517	1.659	1.456
Transform	Translate	-1.078	1.01	6.329	5.555
	Merge	-0.541	0.507	1.592	1.397
	Support	-1.332	1.248	9.662	8.48

association with a certain usecase. To investigate the associations, we calculate Pearson residuals of each observation from the chi-square test. The Pearson residual highlights the most contributing factors towards the final score of a chi-square test, which means in our case, that it would highlight the strength of association between a usecase to proprietary and open-source tools. We present the residual values in Table 4, and for an easier interpretation, a visual representation of the same data in Figure 6. In this figure, the negative residuals are in red indicating lack of support (negative association) between the corresponding usecase and tool type (open-source or proprietary). The figure shows that open-source tools are negatively associated (lack support) for the *Analyze* and *View* category. On the other hand, there is a lack of support between proprietary tools and the usecases *Build*, *Support*, and *Translate*. Similarly, the positive residuals are in blue indicating support (positive association) between the corresponding usecase and tool type (open-source or proprietary). The figure also shows that open-source tools are positively associated to *Build*, *Support*, and *Translate*. Furthermore, proprietary tools moderately support *Analyze* and *View* usecases. The size of the circle accounts for the contribution of a tool type towards a usecase. Since some of the usecases have a positive association with the type of tool (open-source or proprietary), we now investigate the contribution of each type towards a particular usecase in terms of percentage. The residual values proportion for each usecase is presented in Table 4. From the proportions, it can be observed that the most contributing tool-type/use-case residuals to the chi-square results are *View* (23.36%), *Analyze* (14.69%), and *Support* (18.41%). These three usecases contribute 56.46% to the total chi-square result and thus account for most of the difference between open-source and proprietary tools.

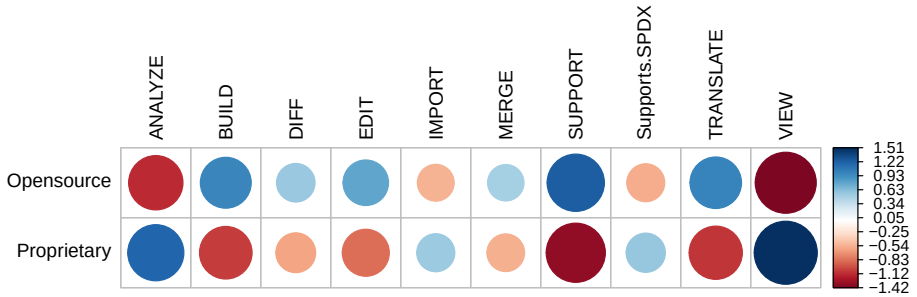


Fig. 6: Visualization of the Pearson residuals to compare the usecases of Opensource .vs. Proprietary SBOM tools.

RQ1: Open-source .vs. Proprietary contribution towards usecases.

We find that the usecase *View* has a strong positive association with Proprietary tools, and the usecase *Support* has a strong positive association with Opensource tools. On the other hand, we find that the usecase *Support* has a strong negative association with Proprietary tools, and the usecase *View* has a strong negative association with Opensource tools. Our results imply to the OSS community that they should focus their attention towards creating tools that can visualize the SBOM for decision-making, while Proprietary tools need to create more libraries and APIs to support integration.

4.2 State of the OSS SBOM tools (RQ2)

In RQ2, we inspect the health of the open-source SBOM tools, we analyze the number of contributors, issues, forks, stars, watchers, pull-requests, releases, and the rate of returning contributors. This analysis will reveal the level of community engagement, activity, and sustainability of open-source SPDX and CycloneDX tools.

4.2.1 Insights on the community-health factors of SBOM tool repositories.

All the open-source community health factors are significantly different from each other in SPDX and CycloneDX. To assess the overall health and activity of the identified tool repositories, we collect their number of stars, forks, releases, contributors, watchers, open/closed issues, and open/closed pull requests. These metrics have previously been used to assess many open-source community health factors of software repositories, such as activeness, maturity, overall maintenance activity to issues, and user engagement of the tool repositories (Zerouali et al., 2019; Foundjem et al., 2021).

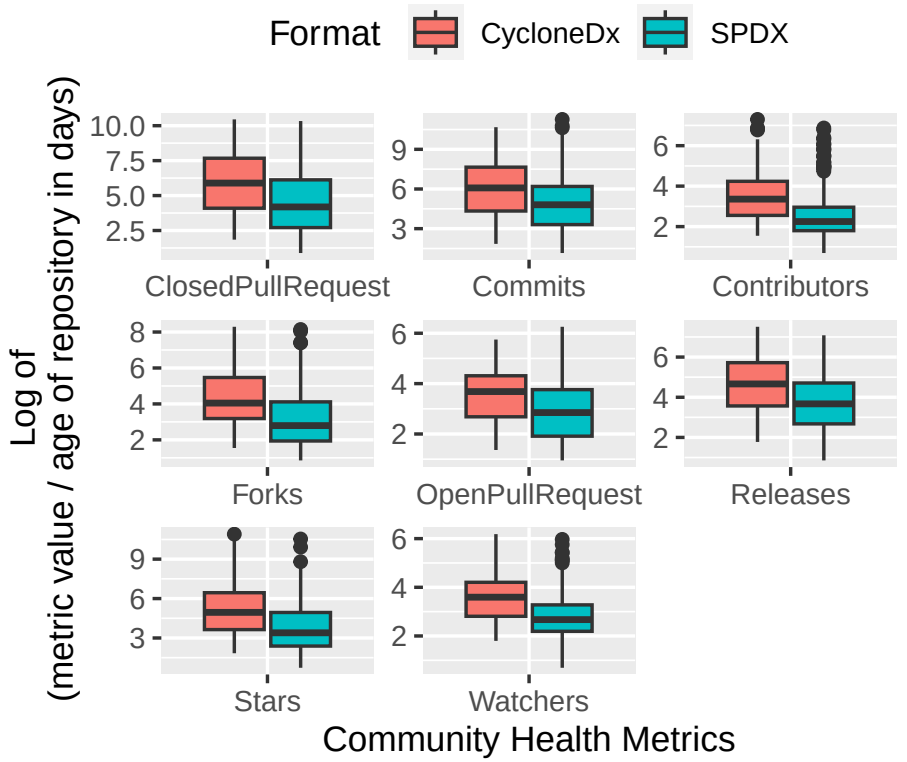


Fig. 7: Metrics, i.e., #PRs, #Contributors, #Stars, #Watchers, #Releases, #Forks representing the community-health metrics of 470 SPDX and 171 CycloneDX tool repositories.

Figure 7 represents the result of what we extracted from the open-source Github tool repositories of 470 SPDX tools and 171 CycloneDX tools. To compare the difference between the tools of the two formats, we applied the Mann-Whitney test (Wilcoxon et al., 1970), on the non-normal distributions of the extracted popularity metric values, followed by the Bonferroni correction to reduce the chance of false positive. The results of this test are available in Table 5, which shows p-value< 0.05 to indicate a significant difference and Cliff's Delta value to indicate the effect size of this difference (Tomczak and Tomczak, 2014).

These results show that all the community-health metrics are significantly different in both formats. Furthermore, the Cliff's Delta effect size suggests that these differences are medium to large.

Since all the metrics significantly differ in both formats and because we have an unequal number of tools from each format, to compare the metrics in the tools of each format, we performed a 1000 samples bootstrapping on the tools with a 95% confidence interval to determine the true median for each metric in both of the formats. We normalize the true median value by the age of each repository to cater the old repositories in our dataset.

Table 5: Mann-Whitney U test (Wilcoxon et al., 1970) applied to evaluate the difference between the community-health metrics of the 171 CycloneDX tool-repositories and 470 SPDX tool-repositories. $p - value < 0.05$ indicates a significant difference between the two distributions.

Metric	p-value	Cliff's Delta	Effect Size (Wan et al., 2019)
ClosedPullRequest	1.00^{-10}	0.595	Large
Commits	7.28^{-07}	0.649	Large
Contributors	2.65^{-15}	0.726	Large
Forks	2.96^{-10}	0.619	Large
OpenPullRequest	8.48^{-10}	0.476	Large
Releases	5.47^{-09}	0.448	Medium
Stars	3.80^{-12}	0.643	Large
Watchers	1.50^{-16}	0.737	Large

[redacted] **CycloneDX tools have more [redacted] of stars, watchers, and forks than SPDX.** Borges and Valente (2018) performed a comprehensive empirical investigation, [redacted] on GitHub projects and found that developers star a project mainly for three reasons: to show appreciation for a project, bookmark a project, or because they are using the project. They further found that 3 out of 4 developers consider the number of stars before contributing to a GitHub project. Finally, they recommend project owners compare the number of stars of their projects with competitors to assess how desirable their project will be to the open-source community. In terms of Stars, CycloneDX tools exhibit higher median 62.3 compared to SPDX tools 5.23. Based on higher number of Stars (Borges and Valente, 2018), our results suggest that open-source developers may desire contributing more to CycloneDX tools than SPDX tools.

Moreover, developers or users watch a repository in order to receive notifications about its activity, such as new issues, pull requests, or discussions. Similar to stars, in terms of watchers, despite having a lesser number of open-source tools than SPDX, CycloneDX median number of watchers 36.2 surpass SPDX 14.4. Number of watchers suggest the rate of observers of projects in open-source community (Borges and Valente, 2018), and we find that CycloneDX tools have far more observers than SPDX tools.

The objective of a fork metric is to ascertain how many copies of a project exist on a code development platform (C. P. L. Foundation, Accessed May 27, 2024). In terms of forks, once again, CycloneDX median number of forks 27.9 surpasses SPDX median number of forks 3.25.

[redacted] **CycloneDX tools have more [redacted] PRs, releases, contributors, and contributions than SPDX.** Pull Requests are a common method for proposing changes to source code repositories, allowing stakeholders to review and discuss proposed changes before integration (Chopra et al., 2021). CycloneDX surpasses SPDX in both the median number of open and closed pull requests. CycloneDX number of open pull requests (median:6.84 and mean:28.2) compared to SPDX (median:0 and mean:10.2), and CycloneDX number of closed pull requests (median:66 and mean:1424) compared to SPDX (median:0 and mean:10.2).

Table 6: Result of Mann-Whitney U Test to compare the difference between returning contributor groups for SPDX and CycloneDX tool repositories.

Groups	p-value	Cliff's delta	Effect size	Significance
1-10	0.000	0.084	negligible	True
10-100	0.423	-0.031	negligible	False
100-1k	0.387	-0.058	negligible	False
1k+	0.710	0.102	negligible	False
all	0.000	0.116	negligible	True

Similarly, CycloneDX tools' number of commits (median:438 and mean:2,453) also surpass SPDX tools' commits (median:122 and mean:1,254).

Number of project releases suggest the level of development, updates, and versioning of a software project. For this metric as well, CycloneDX tools (median:0 and mean:125) surpass SPDX tools (median:0 and mean:26.8).

The number of contributors on GitHub reflects the collaborative nature of a project and the level of community engagement. Similar to the other metrics, in terms of contributors, CycloneDX tools (median:28.2 and mean:88.8) surpass SPDX tools (median:9.25 and mean:34.5).

RQ2: Community-health of tools

The number of star, watcher, and forks suggest that, despite having a lesser number of tools than SPDX, open-source community is more attracted towards the CycloneDX tools. Number of stars further suggests that open-source developers may prefer contributing to CycloneDX tools than SPDX tools because CycloneDX tools have a higher number of stars (Borges and Valente, 2018). Similarly, all the metrics: PullRequests, Issues, Contributors, and Releases; suggest that, despite having a lesser number of tools than SPDX, CycloneDX tools show a higher development activity than SPDX tools.

[redacted] **There is a negligible difference between the rate of returning contributors of CycloneDX tools and SPDX tools.** A returning contributor of a project is a Github contributor who contributes to a project more than once. We group the returning contributors by their number of contributions (commits) and compare them between the two formats (SPDX and CycloneDX) to investigate which format tools have a better rate of returning contributors. Table 6 presents the Mann-Whitney U test statistics for the difference of SPDX and CycloneDX tools across the returning contributor groups (by number of commits), including Bonferroni corrected p-values, Cliff's delta, effect sizes, and significance level of p-value. For the small commit count group ($1 - 10$ commits), the p-value is 0.000, indicating a significant difference between SPDX and CycloneDX. However, the effect size (Cliff's delta) is 0.084, which is considered negligible. For the medium commit count group (10-100 commits), the p-value is 0.423, showing no significant difference between SPDX and CycloneDX. The effect size is -0.031, also negligible.

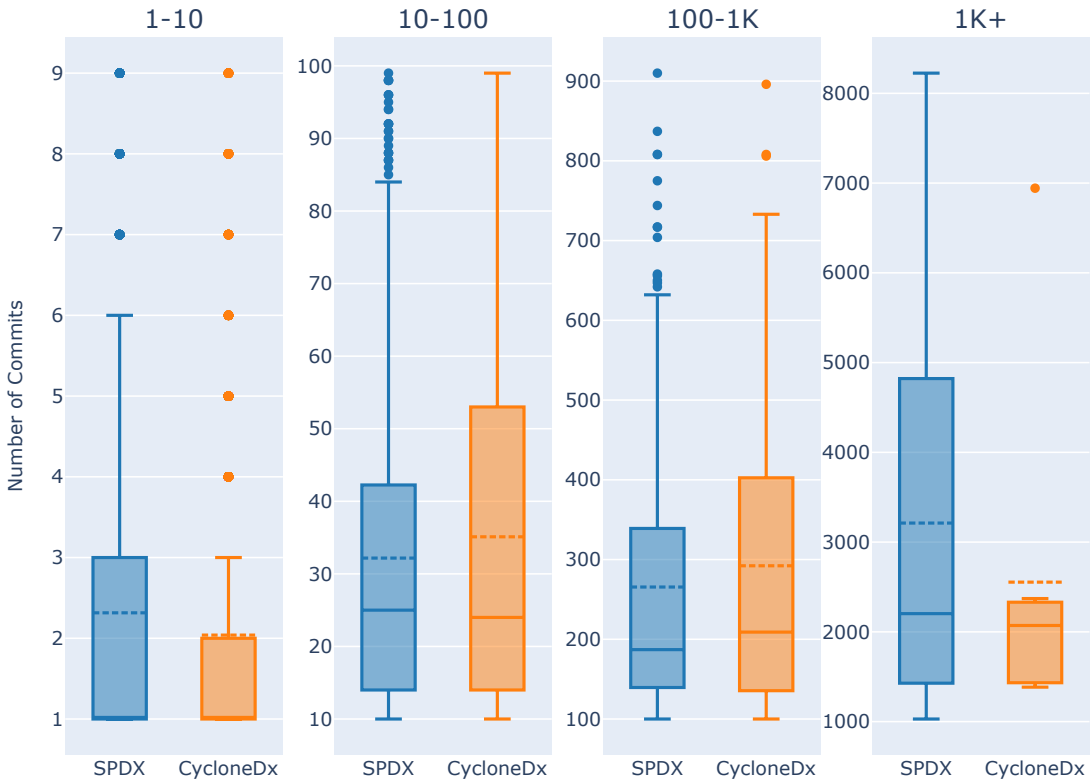


Fig. 8: Distribution of open-source contributor groups (by number of contributions) across 470 SPDX and 171 CycloneDX tool repositories.

For the high commit count group ($100 - 1,000$ commits), the p-value is 0.387, indicating no significant difference between SPDX and CycloneDX. The effect size is 0.058, negligible as well. For the extremely high commit count group ($1,000+$ commits), the p-value is 0.710, showing no significant difference between SPDX and CycloneDX. The effect size is 0.102, negligible. When considering all commit count groups combined, the p-value is 0.000, indicating a significant difference between SPDX and CycloneDX. However, the effect size is 0.116, which is still negligible.

To further illustrate these findings, Figure 8 shows the distribution of open-source contributors across various commit count groups for SPDX and CycloneDX tool repositories. Compared with the two, the median and mean number of commits for SPDX (1 and 2.32) and CycloneDX (1 and 2.04) are similar for small commit counts ($1 - 10$), indicating a slightly higher commitment from returning contributors in SPDX. Similarly, for medium commit counts ($10 - 100$), both SPDX (median: 25, mean: 32.18) and CycloneDX (median: 24, mean: 35.11) exhibit a comparable level of commitment among returning contributors. However, for high counts of commits ($100 - 1k$), CycloneDX (median: 209, mean: 292.18) shows a lower level of commitment than SPDX (median: 187, mean: 265.52). Fi-

nally, for the highest commit counts ($1k+$), SPDX shows a higher commitment level (median: $2,203$, mean: $3,212.05$) than CycloneDX (median: $2,071$, mean: $2,554$).

RQ2: Returning Contributors

From the outlook, SPDX tools, when compared to CycloneDX tools, attract a higher level of commitment from its contributors with $1,000+$ commits, and similarly, attract a slightly higher level of commitment from contributors with a small number of commits ($1 - 10$ commits). In contrast, CycloneDX show more returning contributors in the range of $10 - 1000$ commits. However, the statistical test comparison concludes tools from both of the formats have negligible difference when it comes to the number of returning contributors.

4.3 Issues in the SBOM OSS tools (RQ3)

Since previous studies have shown the potential of uncovering developers' concerns and challenges from mining Github issues (Kikas et al., 2015; Batoun et al., 2023), in this RQ, we study the prevalence of GitHub issues in the open-source SBOM tools that support the CycloneDX and SPDX formats. Issues in Github repositories are used to report, discuss, and track work within a repository. Knowing the resolution time for a specific issue category would draw the attention of SBOM developers to the tools that need improvement. Furthermore, insights into the kind of issues will enable SBOM practitioners to make informed decisions about tool selection and implementation in their supply chain. Furthermore, developers can use this feedback to prioritize bug fixes, address user needs, and enhance the overall functionality of SBOM tools.

4.3.1 Comparing closed and open issues.

CycloneDX tools have a higher number of mean issues than SPDX tools. Number of issues, whether open or closed, can indicate the level of activity, responsiveness to bug reports or feature requests, and overall project health (Kikas et al., 2015). We compare the CycloneDX and SPDX tool repositories by the number of issues using the Mann-Whitney U test and Bonferroni correction applied, and find a statistically significant difference between the tools of both formats, with p-value=0.0003 and Cliff's Delta effect size of 0.618 (Large). For a fair comparison, in this RQ, we consider only those issues that were created after 2018 because CycloneDX technology was released in May 2017. We find that CycloneDX tools have a higher mean average for having number of issues than SPDX tools, in both open and closed issue categories. Furthermore, after applying a 1000 samples bootstrapping with a 95% confidence interval, we still find that CycloneDX have more mean number of issues (lower bound:53.9 and upper bound: 150.1) than SPDX (lower bound:34.01 and upper bound: 85.89).

CycloneDX tools have a higher number of open and closed [redacted] issues than SPDX tools. We then group the issues by their status: open and

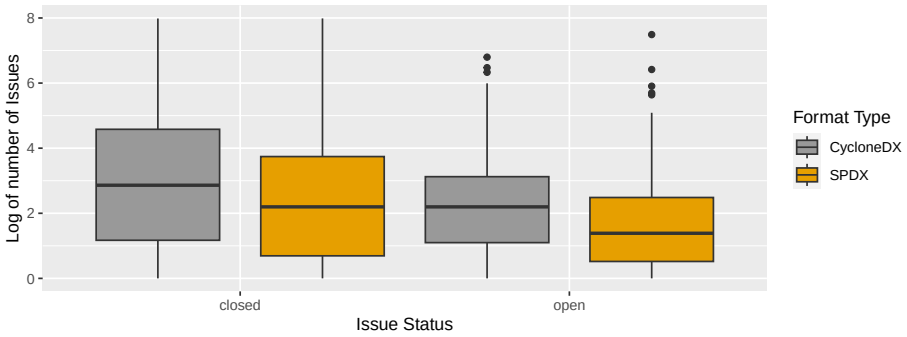


Fig. 9: Representation of 17,093 issues in 90 CycloneDX tool repositories and 16,747 issues in 171 SPDX tool repositories.

closed. On the issue status level, Figure 9 shows 4,974 open issues in 171 SPDX tools compared to 4,449 open issues in 90 CycloneDX tools. The figure also shows 11,773 closed issues in SPDX tools compared to 12,644 closed issues in CycloneDX tools. We apply the Mann-Whitney U test with Bonferroni correction to compare the tools by number of open and closed issues. In open issues, by prevalence, we found a statistical difference with a p-value=0.0039 with Cliff's Delta effect size 0.586 (Large), and in closed issues we find a significant difference with a p-value=0.0207 with Cliff's Delta effect size 0.572 (Large). We find that the mean number of open issues in CycloneDX is 54.25 with a standard deviation of 141.11, which surpasses the mean number of SPDX open issues 38.55 with a standard deviation of 161.87. Similarly, the mean number of closed issues in CycloneDX is 162.10 with a standard deviation of 441.07 and the mean number of closed issues in SPDX is 85.31 with a standard deviation of 258.46. A higher standard deviation of SPDX tools in both open and closed issues suggests inconsistency in issue activity in SPDX tools compared to CycloneDX.

RQ3: Issue Activity

CycloneDX tools community is more active in terms of issue creation (33.96% more open issues than SPDX tools) and resolution (92.50% more closed issues than SPDX tools) than SPDX, with a significant difference and large effect size.

4.3.2 Comparing resolution time of the issues.

CycloneDX tools have a faster and more stable resolution time for their Github issues than SPDX. We investigate if the tools of one format have a faster response time for resolving the issues when compared to the tools of another format. To compare the resolution time of the issues across the two formats, we apply the Mann-Whitney U test with Bonferroni correction and found that both formats' issue resolution time significantly differ from eachother with $p - value =$

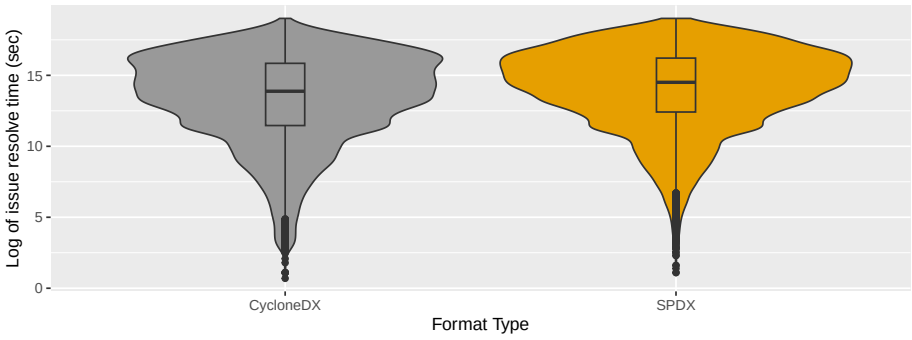


Fig. 10: Resolution time of 13,281 closed issues in 90 CycloneDX tool repositories and 13,352 closed issues in 191 SPDX tool repositories.

$1.32e-60$. Figure 10 presents the resolve time of both formats' tools, we observe that, in terms of mean resolution time, despite having more number of issues than SPDX, CycloneDX still has 67.69% better resolution time than SPDX. SPDX has an average resolution time of 130.74 days, while CycloneDX has an average resolution time of 88.12 days. The standard deviation of SPDX tools resolution time is also 71% higher than CycloneDX. Having better resolution time and lesser standard deviation suggests that CycloneDX OSS tools community is more active towards resolving the Github issues (Kikas et al., 2015).

RQ3: Issue Activity

Despite having more number of issues than SPDX tools, CycloneDX tool issues are resolved much more quickly which suggests an active tool community compared to SPDX.

4.3.3 Trend analysis of the issues.

CycloneDX tools have more issues and better resolution time since 2021. Since there is a significant difference in the overall number of OSS tool issues in SPDX and CycloneDX, we further compare them to analyze the recent trend, i.e., of the past few years (from 2018 to 2023). Figure 11 suggests that until 2020 SPDX was having more number of issues, however, CycloneDX always had a better resolution time. [redacted] after 2020, CycloneDX community has been more active SPDX with more number of issues and better resolution time.

(a) Prevalence of issues.

(b) Resolution time of issues.

Fig. 11: Trend of prevalence and resolution time of issues from 2018 to 2023.

RQ3: Recent Trends

Since 2021, i.e., the same year when US government issued an executive order mandating the adoption of Software Bill of Materials (SBOMs) on companies who want to conduct business with the US (Remaley and Remaley, 2021), CycloneDX tools-community has been more active in resolving issues than SPDX.

4.3.4 Tags analysis of the issues.

SPDX tools have most of the issues related to "Feature Development and Enhancement" and "Code Components", while CycloneDX tools have more issues related to "Bug Fixes and Defects" and "Feature Development and Enhancement". Understanding the type of issues in each SBOM format would help tool developers improve their tools and help SBOM users make the right decision when selecting an SBOM tool for their project. We identify 14 distinct categories from the issues collected from the 87 SPDX and 61 CycloneDX tool repositories, using the manual tag classification approach described in Section 3.4. We have listed the indices and names of each category, accompanied by a short description, the set of relevant tags, and one representative example as follows.

C_{01} **Bug Fixes and Defects**
Definition: This category refers to technical problems within the codebase **Tags**: exceptions, stale, false-positive, defect, bug, kind/bug, technical issue, stale, good-first-issue, lifecycle/stale, lifecycle/frozen, QA **Example**: "PolicyExceptions not being applied to EphemeralContainers" in the "kyverno/kyverno" repository[11].
C_{02} **Code Quality**

[11] `https://github.com/kyverno/kyverno/issues/9484`

Definition: This category refers to improving the overall quality of code.
Tags: refactor, comment-styles, log enhancement, changelog-ignore, package-formats, package scan, performance, validation, generation
Example: "Missing CVEs on openssh" in the "anchore/grype" repository[12].

C_{03} **Community Engagement and Support**

Definition: This category refers to engaging the community, seeking feedback, and providing support to contributors.
Tags: request for comment, upstream, help wanted
Example: "Add bom-refs to OrganizationalEntity and OrganizationalContact" in the "CycloneDX/specification" repository[13].

C_{04} **Configuration**

Definition: This category refers to configuring, packaging, and managing dependencies for the project.
Tags: configuration, docker, helm, installation and packaging, dependencies
Example: "Maximize portability by using fallback pure-python libraries" in the "nexB/scancode-toolkit" repository[14].

C_{05} **Continuous Integration and Deployment**

Definition: This category refers to setting up and maintaining continuous integration and continuous deployment pipelines.
Tags: CI, Type: CI, CI/CD
Example: "Change logo in gh-pages" in the "linterhub/schema" repository[15].

C_{06} **Documentation**

Definition: This category refers to creating and maintaining documentation for the project.
Tags: docs, Documentation, documentation
Example: "review the license type detection" in the "CycloneDX/cyclonedx-node-module" repository[16].

C_{07} **Feature Development and Enhancement**

Definition: This category refers to adding new features and enhancements to the project.
Tags: kind/feature, proposed core enhancement, should have, nice to have, feature, must have, proposal, enhancement, enhancement, new feature, security
Example: "Feature request: Map k8s pod labels as project tags" in the "ckotzbauer/sbom-operator" repository[17].

C_{08} **Integration**

Definition: This category refers to integrating with various operating systems and ecosystems.

[12] https://github.com/anchore/grype/issues/1695
[13] https://github.com/CycloneDX/specification/issues/226
[14] https://github.com/nexB/scancode-toolkit/issues/3210
[15] https://github.com/linterhub/schema/issues/174
[16] https://github.com/CycloneDX/cyclonedx-node-module/issues/234
[17] https://github.com/ckotzbauer/sbom-operator/issues/172

Tags: os support, ecosystem:javascript, ecosystem:java, webhook
Example: "scancode-toolkit-mini 30.1.0 - missing dependency" in the "nexB/scancode-toolkit" repository[18].

C_{09} **Libraries**

Definition: This category refers to developing and managing of libraries and APIs.
Tags: spdx-utils, API Call, core and api
Example: "How to provide custom license texts?" in the "oss-review-toolkit/ort" repository[19].

C_{10} **Licensing**

Definition: This category refers to copyrights and licensing.
Tags: improve-license-detection, copyright scan, Profile:Licensing, license-review, improve-license-detection, Submit New License, new license/exception: Not Accepted, new license/exception: Accepted, new license/exception request
Example: "New license request: AzFOSS" in the "spdx/license-list-XML" repository[20].

C_{11} **Code Components**

Definition: This category refers to specific components of the codebase, such as serialization, parsers, and package scanning.
Tags: serialization, data model, parser, analyzer, scanner, I/O, package-at-toplevel, package-files, package-formats, package scan, component:dep-sources
Example: "Integrate DWARF plugin into SCTK release and correct documentation" in the "nexB/scancode-toolkit" repository[21].

C_{12} **Release Management and Versioning**

Definition: This category refers to managing releases, handling breaking changes, and versioning of the project.
Tags: breaking change, release-high, pending release, breaking-change, breaking change
Example: "Remove deprecated configuration" in the "anchore/syft" repository[22].

C_{13} **User Interface and Outputs**

Definition: This category refers to the user interface and outputs of the project.
Tags: GUI and outputs, type:cli, end user
Example: "Incorrect progress bar display" in the "nexB/scancode-toolkit" repository[23].

C_{14} **Technical Debt**

Definition: This category refers to addressing the technical debt of the project.
Tags: technical-debt, technical debt

[18] `https://github.com/nexB/scancode-toolkit/issues/2752`
[19] `https://github.com/oss-review-toolkit/ort/issues/4740`
[20] `https://github.com/spdx/license-list-XML/issues/1365`
[21] `https://github.com/nexB/scancode-toolkit/issues/2422`
[22] `https://github.com/anchore/syft/issues/1864`
[23] `https://github.com/nexB/scancode-toolkit/issues/389`

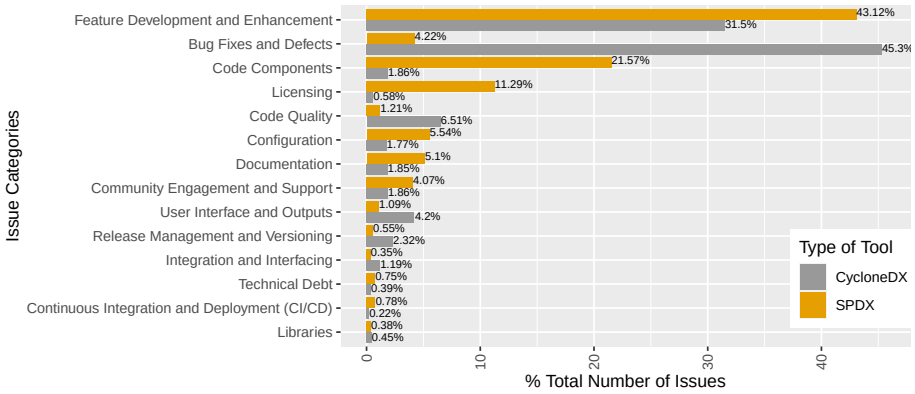


Fig. 12: Issue categories prevalence across SPDX and CycloneDX tools. *[Ali says: WORKING ON THE SYMMETRIC PLOT.]*

Example: "Refactor analyze_docker_image to reduce complexity" in the "tern-tools/tern" repository[24].

Figure 12 illustrates the prevalence of each issue category. For SPDX, issues related to "Feature Development and Enhancement" are the most prevalent (43.12%), closely followed by "Code Components" (21.57%) and "Licensing" (11.29%). In contrast, CycloneDX exhibits a different distribution, with "Bug Fixes and Defects" being the most prevalent (45.3%), followed by "Feature Development and Enhancement" (31.5%) and "Code Quality" (6.51%). These findings highlight the distinct challenges and priorities that each SBOM format faces in its respective tool ecosystems.

Table 8: Result of Mann-Whitney U Test to compare the difference between the resolution time of issue categories by CycloneDX.vs. SPDX.

Category	**p-value**	**Cliff's delta**	**Effect size**
Bug Fixes and Defects	2.2e-16	0.278	Small
Code Quality	0.257	-	-
Code Components	0.8705	-	-
Community Engagement and Support	0.1181	-	-
Configuration	0.0527	-	-
Continuous Integration and Deployment (CI/CD)	0.03588	0.631	Large
Documentation	0.054	-	-
Feature Development and Enhancement	2.2e-16	0.387	Medium
Integration and Interfacing	0.0088	0.342	Medium
Libraries	0.07501	-	-
Licensing	0.0448	0.581	Large
Release Management and Versioning	0.3662	-	-
Technical Debt	0.4418	-	-
User Interface and Outputs	0.0077	0.4008	Medium

[24] `https://github.com/tern-tools/tern/issues/212`

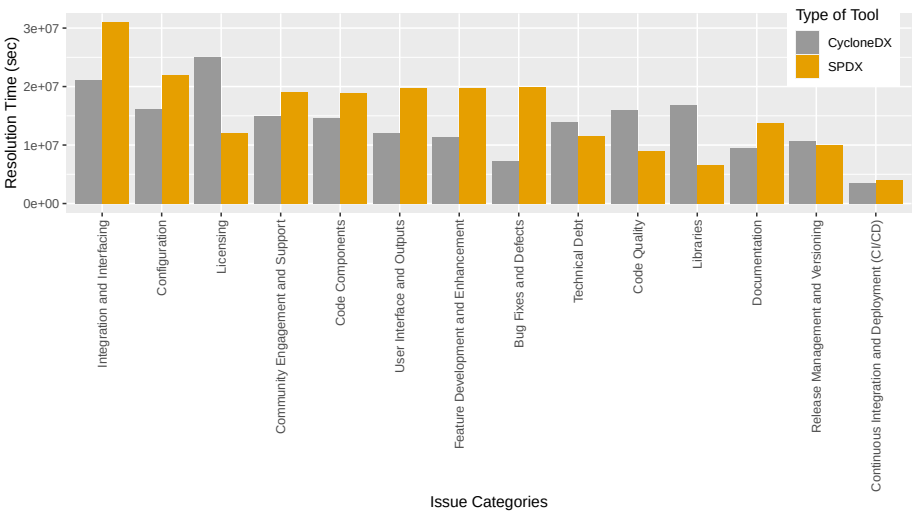


Fig. 13: Resolution time (in seconds) of issue categories of SPDX and CycloneDX tools. *[Ali says: WORKING ON THE SYMMETRIC PLOT.]*

Amongst the issue-categories, SPDX tools resolves issues related to Licensing 51.85% faster than CycloneDX, while CycloneDX tools resolves other issue-categories (the ones that have significantly different resolution time in SPDX) 78.87% faster than SPDX. To further understand which format spends how much time resolving what type of issues, we determine the resolution time of each issue category in a format, as presented in Figure 13. We then compare the resolution time of the categories within each format. For this comparison, we apply the Mann-Whitney U test of statistical difference and Cliff's Delta for effect size. We find significant difference in the following categories: "Bug Fixes and Defects" with small effect size, "Continuous Integration and Deployment (CI/CD)" with large effect size, "Licensing" with large effect size, "Integration and Interfacing" with medium effect size, "User Interface and Outputs " with medium effect size, and "Feature Development and Enhancement" with medium effect size. The results of the statistical tests can be observed in Table 8. CycloneDX resolves issues 174.03% faster in the "Bug Fixes and Defects", 15.9% faster in the "Continuous Integration and Deployment (CI/CD)", 73.41% faster in the "Feature Development and Enhancement", 46.52% faster in the "Integration and Interfacing", and 64.47% faster in the "User Interface and Outputs " category. Whereas, SPDX resolves issues that are related to "Licensing" 51.85% faster than CycloneDX.

Table 9: Mann-Whitney U Test (Wilcoxon et al., 1970) applied to evaluate the difference between the project characteristics and popularity metrics of 250 CycloneDX employing projects and 250 SPDX employing projects. $p - value < 0.05$ indicates a significant difference between the two distributions.

Metric	p-value	Cliff's Delta	Effect size
ClosedIssues	3.79^{-12}	0.666	Large
ClosedPullRequest	2.74^{-09}	0.655	Large
Contributors	0.07^{-01}	0.576	Large
Forks	2.50^{-16}	0.709	Large
OpenIssues	5.39^{-09}	0.618	Large
OpenPullRequest	0.03^{-02}	0.525	Large
Releases	0.43^{-01}	0.592	Large
Stars	6.09^{-27}	0.783	Large
Watchers	3.35^{-12}	0.668	Large
Commits	3.38^{-06}	0.632	Large

RQ3: Most prevalent issues

SPDX tools have most of the issues related to "Feature Development and Enhancement", while CycloneDX tools have more issues related to "Bug Fixes and Defects". Among the categories that have significantly different resolution times, SPDX tools resolves issues related to Licensing 51.85% faster than CycloneDX, while CycloneDX tools resolves the remaining issue-categories 78.87% faster than SPDX.

4.4 Prevalence of SBOM tools in OSS projects (RQ4)

In this RQ, we investigate those Github projects that carry the create command from SBOM tool in their build configuration or code. By analyzing these open-source projects, we provide insights into the developer community trends and the characteristics of the projects that use SBOM tools (SPDX or CycloneDX). We further calculate the demand of SBOM features in the open-source community. SPDX and CycloneDX tool developers may use this information to improve the features of their tools.

4.4.1 Insights on the developer community of OSS projects that use SBOM tools.

To understand the significance of the projects that use the SPDX and CycloneDX tools, we assess the community health of the identified projects through the same metrics that were used in RQ2 – # of Stars, Watchers, Forks, Issues, Contributors, Pull Requests, Releases, and Commits. These metrics have previously been used in literature to assess many quality factors of software repositories, such as consistent documentation, number of downloads, popularity, and development activity Zerouali et al. (2019). These metrics also provide an idea about the health of the open-source community around these projects C. P. L. Foundation (Accessed May 27, 2024).

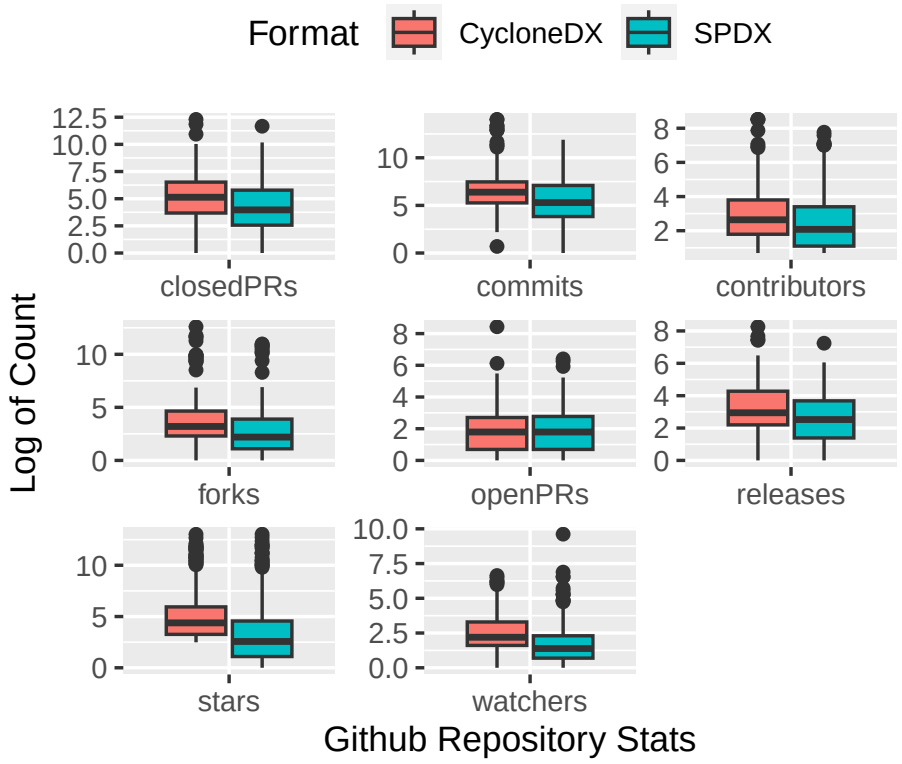
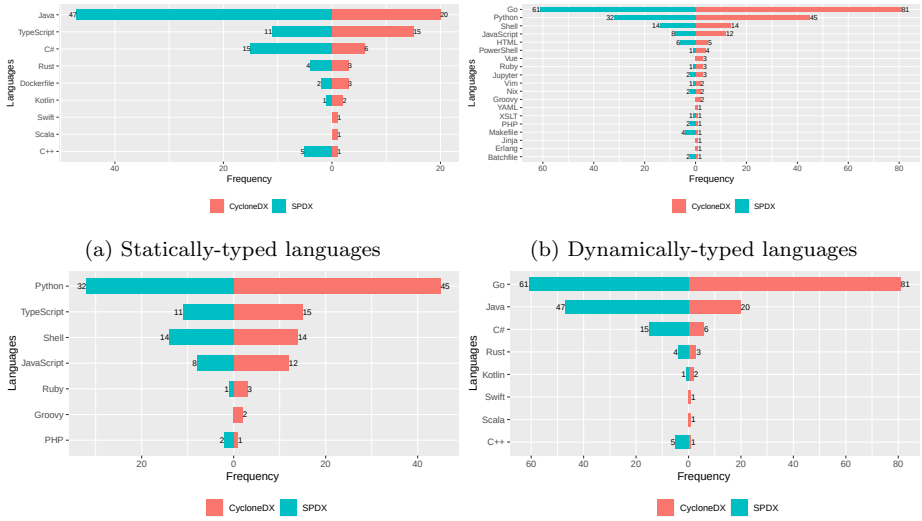


Fig. 14: Community health metrics of Top-250 projects that use SPDX tools .vs. Top-250 projects that use CycloneDX tools. Log of count is normalized by the age of repository in days × 100.

CycloneDX outperforms SPDX in all the open-source community health factors, such as stars, watchers, forks, PRs, releases, contributors, and commits. Figure 14 presents the values of the metrics of the project repositories that employ CycloneDX or SPDX tools. To compare the difference between the SPDX-using and CycloneDX-using projects, we applied the Mann Whitney-U test (Wilcoxon et al., 1970) followed by the Bonferroni correction, on the non-normal distributions of the extracted metric values. Table 9 shows test results and the respective effect sizes(Cohen, 1992). It can be observed from this table that both [redacted] sets are significantly different from each other for all the metrics, with a "large" effect size too.

Projects using CycloneDX tools have a higher median value (normalized by age of repositories in days) in terms of stars (0.80 in CycloneDX vs. only 0.04 in SPDX), forks (0.22 in CycloneDX vs. only 0.04 in SPDX), and watchers (0.08 in CycloneDX.vs. 0.033 in SPDX). Number of stars (also known as stargazers) is considered an indicator of admiration and a positive attitude towards the chosen repository, which can indicate higher trust and usage Borges and Valente (2018); and CycloneDX adopting projects beats SPDX adopting projects in this metric.



(a) Statically-typed languages

(b) Dynamically-typed languages

(c) Scripting languages

(d) Procedural languages

Fig. 15: Programming language of projects that use CycloneDX tools .vs. projects that use SPDX tools.

Furthermore, CycloneDX-using projects having a higher number of forks indicating an active contributing community compared to SPDX-using projects (Jiang et al., 2017) and CycloneDX-using projects have a higher number of watchers indicating greater observability by the OSS community compared to the SPDX-using projects (Borges and Valente, 2018).

Repositories using CycloneDX tools have a notably higher issue activity both in terms of median (normalized by age of repositories in days) open pull request (0.03 in CycloneDX vs. 0.01 in SPDX), median closed pull request (1.57 in CycloneDX vs. 0.21 in SPDX), median releases (0.19 in CycloneDX vs. 0.12 in SPDX), median contributors (0.14 in CycloneDX vs. 0.08 in SPDX), and median commits (5.94 in CycloneDX vs. 2.00 in SPDX). This trend suggests that CycloneDX projects receive more attention and effort from the OSS community. In addition, greater attention to pull requests and commits reflects the contribution of OSS to projects (Jarczyk et al., 2014).

RQ4: Community-health of projects that use SBOM tools

All the community-health metrics suggest that Top-250 CycloneDX-using projects have a healthier OSS community than Top-250 SPDX-using projects, with a large effect size on the difference between the two formats.

4.5 Insights on the language distribution of OSS projects that use SBOM tools.

Projects written in procedural languages prefer SPDX tools, while projects written in scripting languages prefer CycloneDX tools. The choice of the programming language for a project significantly impacts development efficiency, performance, maintainability (Bhattacharya and Neamtiu, 2011), security, and code quality (Ray et al., 2017). To understand which programming language is preferred by which project (CycloneDX-using or SPDX-using), we classify the projects based on programming paradigm, i.e., whether a project is written in a procedural, functional, or scripting language.

Figure 15 shows the language distribution of projects into procedural and scripting languages. In Figure 15, it can be observed that a higher proportion of projects utilizing scripting languages leaning towards CycloneDX tools, whereas procedural languages exhibit a greater presence in projects that use SPDX tools. Procedural languages, including Go, Java, and C#, are commonly used in system-level programming, performance-critical tasks, or the development of large-scale applications, indicating a preference for SPDX tools in such projects. Conversely, scripting languages such as Python, TypeScript, and Shell Scripting are favoured for web development, automation, or data processing, indicating a preference for CycloneDX tools in these projects.

RQ4: Language preference

Projects that use CycloneDX tools show a higher usage of scripting languages, while projects that use SPDX tools show a higher usage of procedural languages. This information can help selecting the most suitable tools for a project based on its primary language. It also aids in tailoring training and resources for developers to align with the tools and languages they are most likely to encounter in their projects.

4.6 Tool preference across different SBOM projects

The choice of an SBOM tool varies depending on the number of commits of a project. We determine if different projects prefer using tools that support different usecases. For this analysis, we first group the projects into four quartiles based on their number of contributors. We applied Chi-square and found that there is a statistical difference among the quartiles (p-value = $2.29e^{-20}$, however, with a weak effect (Cramér's V = 0.095). We then group the projects into four quartiles based on their number of commits and find a statistical difference among the quartiles (p-value = $1.65e^{-40}$) with a moderate effect size (Cramér's V = 0.124) Cramér (1999). We present the distribution of chi-square residual values in Figure 16. Lower values in this figure indicate lesser demand for a usecase, while higher value indicate higher demand for a usecase. The size of the bubble represents the proportion of a quartile usecase that contributed towards the chi-square's significance.

Our analysis reveals that *View*, *Diff*, and *Import* usecases exhibit larger Pearson residuals for the last two quartiles (Q3 and Q4), suggesting that projects

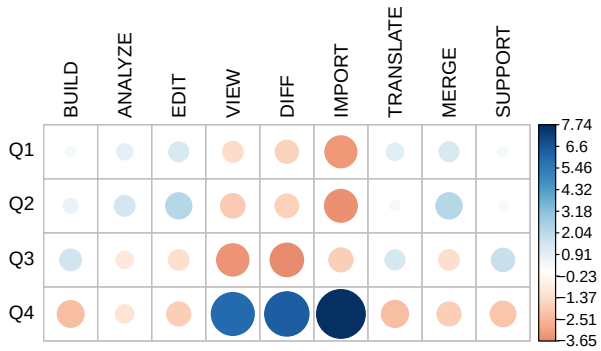


Fig. 16: Visualization of the Pearson residuals to compare the Quartiles (by number of commits) with the usecases of the tools they use. The quartiles are normalized by the age of repository in days. Q1 = 0.00029–0.042, Q2 = 0.043–0.16, Q3 = 0.17–0.055, Q4 = 0.56–2508.56. The size of the bubble denotes the significance of a particular usecase towards the chi-square result.

having larger number of commits prefer tools with *View*, *Diff*, and *Import* usecases. Furthermore, *Analyze*, *Edit*, and *Merge* are more prevalent in the first two quartiles (Q1 and Q2), suggesting that projects having lesser number of commits prefer tools with *Analyze*, *Edit*, and *Merge* usecases.

RQ4: Tool preference across different SBOM projects

Large projects (by number of commits) prefer tools that support *View*, *Diff*, and *Import* usecases, while smaller projects have lesser demand for such tools. Similarly, smaller projects are more inclined towards tools that support *Edit*, and *Merge*, while larger projects have lesser demand for such tools.

5 Implications

Improvement of Open-Source SBOM Tools. The study sheds light on the strengths and limitations of open-source SBOM tools compared to proprietary ones. By pinpointing specific use cases where open-source tools underperform, developers can focus their efforts on improving these areas, thereby bridging the gap between open-source and proprietary tools and making open-source options more versatile and competitive across various applications.

Enhanced Decision-Making for SBOM Tool Adoption. The insights from this study equip practitioners and policymakers with the knowledge needed to make informed choices when selecting SBOM tools. A clear understanding of the strengths and weaknesses of SPDX and CycloneDX formats enables organizations to choose the most suitable tool based on their unique requirements, such as integration with build processes or support for multiple formats. This helps facilitate strategic planning and the adoption of the best-fit tools within the software supply chain.

Identification of Prevalent Issues for Targeted Solutions. The analysis of commonly reported issues in open-source SBOM tools provides a comprehensive understanding of the challenges faced by the tools of each technology. By categorizing these issues and comparing the resolution times between SPDX and CycloneDX tools, the study offers actionable insights for developers to prioritize and address the most critical problems. This targeted approach to issue resolution may significantly enhance the usability and reliability of SBOM tools, ultimately fostering more secure and efficient software supply chains.

Enhancement of Community Health and Engagement. Evaluating the community health of open-source SBOM tools using CHAOSS metrics helps identify which tools are more actively maintained and adopted. This information guides open-source developers to contribute to high-demand tools, improving their functionality and user support. Moreover, it assists practitioners in selecting tools that are actively developed and maintained, ensuring long-term sustainability and reliability within their organizations.

6 Threats to Validity

6.1 Conclusion Validity

This study focuses on the tool ecosystem of the two most popular SBOM formats, SPDX and CycloneDX. As such, our findings are specific to these formats and cannot be generalized to other SBOM technologies, such as Software Identification Tags (SWID) (NTIA, 2021b). Future research could explore the performance and community health of other SBOM formats and tools to provide a more comprehensive understanding of the broader SBOM ecosystem.

6.2 Construct Validity

In RQ4, we did not configure or execute the GitHub projects to verify the generation of SBOM files. Instead, we focused on identifying the presence of SBOM generation patterns in their code or build configuration files. This approach allowed us to efficiently analyze a large number of projects, but did not account for the actual implementation or effectiveness of these patterns. Additionally, our study does not investigate the real world adoption or usage of SPDX and CycloneDX formats in the industry, which could offer further insights into their practical importance and influence. Future research may consider addressing these aspects to provide a more comprehensive understanding of the impact and adoption of various SBOM formats.

While there is another SBOM format called SWID, we chose not to include it in our study, as it is considered more a software identifier than a comprehensive SBOM format, such as SPDX and CycloneDX (Brudo, 2024). SWID tags provide a simple method to track software inventory, but lack the depth and analysis capabilities offered by SPDX and CycloneDX, which include detailed information on software components, licenses, vulnerabilities, and other metadata (Das, 2023). Comparison of SWID tool ecosystems with SPDX and CycloneDX would not yield equivalent insights due to the fundamental differences in their scope and purpose.

6.3 Internal Validity

SPDX and CycloneDX support various file formats, but only JSON and XML are common to both. Although each format could potentially cover different SBOM documentation aspects such as licensing and security, we conducted a manual ad hoc comparison of the JSON and XML specifications provided by each technology. Our analysis suggested that both JSON and XML specifications cover the same SBOM documentation features. Based on this finding and for simplicity, we decided to focus solely on the XML specification for the RQ1 analysis. We believe that this decision does not compromise the validity of our results, as both file formats appear to support equivalent features of the SBOM documentation. Future research could explore the potential differences between file formats in more depth to validate this assumption.

6.4 External Validity

In RQ3, our analysis relies on the issues reported in the GitHub repositories of the CycloneDX and SPDX tools. As a result, the findings may not apply to other SBOM formats or repositories hosted on platforms other than GitHub. To minimize this limitation, we focus on the two most popular SBOM technologies with a substantial user base, ensuring a broad and representative view of their ecosystems. However, it is important to note that our study does not capture all CycloneDX and SPDX-related discussions, issues, or contributions that occur outside of GitHub. While we believe our approach provides valuable insights, future research could explore additional sources of information to further enrich our understanding of these ecosystems.

Focusing on tools available on the official CycloneDX and SPDX websites aimed to streamline our data collection process by leveraging expert-curated lists. This approach increases the likelihood of identifying functional tools, as they have been vetted by the respective communities. Additionally, it reduces the risk of including prototype tools that may not be suitable for practical use. However, this strategy may inadvertently exclude functional, non-prototype tools hosted elsewhere. While the likelihood of this being relatively low, future research could expand the scope to include a broader range of sources, ensuring a more comprehensive overview of the available tools. Nonetheless, our current approach offers a solid foundation for understanding the landscape of CycloneDX and SPDX tools.

Our analysis is based on a specific point in time snapshot of the SPDX and CycloneDX ecosystems. As these ecosystems continue to grow and evolve, the distribution of tools and issue categories can change, and new trends can surface. While our findings provide valuable insights into the current state of these ecosystems, ongoing monitoring and evaluation will be necessary to capture emerging trends and updates in the dynamic ecosystem of SPDX and CycloneDX tools.

7 Conclusion

This study provides a comprehensive analysis of the SBOM tools ecosystem, comparing the two predominant SBOM formats: SPDX and CycloneDX. Through

qualitative and quantitative analyses of 108 open-source and 62 proprietary SBOM tools, as well as an in-depth investigation of the community health metrics of the 641 tools and the characteristics of top open-source projects, we have obtained several key insights.

Firstly, we observed that proprietary tools generally offer a broader range of use cases compared to open-source SBOM tools, particularly in the areas of build automation, artifact analysis, and visualization. This indicates a need for the open-source community to enhance the functionalities of their SBOM tools to bridge this gap. Secondly, our findings on community health metrics reveal that CycloneDX tools tend to have higher developer engagement, as evidenced by their superior value in stars, watchers, forks, pull requests, and releases. This suggests a more vibrant and active community around CycloneDX tools, which could influence their adoption and sustainability in the long run. Moreover, the analysis of GitHub issues highlighted the prevalent challenges faced by both the SPDX and CycloneDX tools. While the CycloneDX tools showed a faster issue resolution time, the SPDX tools excelled in areas, such as community engagement and support. This information is crucial for developers and contributors to focus their efforts on addressing the specific needs and challenges associated with each format. Finally, our study of GitHub projects that initialize SBOM tools in their code showed distinct preferences and adoption patterns. Projects using CycloneDX tools demonstrated better health metrics, with a higher number of stars, watchers, releases, pull requests, and contributors. Furthermore, the study found that dependency projects with varying levels of recurring contributions exhibit distinct preferences for SBOM operations.

In conclusion, while both SPDX and CycloneDX formats have their unique strengths and weaknesses, CycloneDX tools appear to be more popular and actively maintained within the open source community. For practitioners and policymakers, this study provides valuable insights into the current state of SBOM tools, guiding them in making informed decisions regarding tool selection and implementation to improve software supply chain security. We identify areas for improvement for SBOM tool developers and offer a roadmap to enhance the functionalities of their tools and community support, ultimately contributing to the development of a robust SBOM tools ecosystem.

References

Balliu M, Baudry B, Bobadilla S, Ekstedt M, Monperrus M, Ron J, Sharma A, Skoglund G, Soto-Valero C, Wittlinger M (2023) Challenges of producing software bill of materials for java. `2303.11102`

Batoun MA, Yung KL, Tian Y, Sayagh M (2023) An empirical study on github pull requests' reactions. ACM Transactions on Software Engineering and Methodology

Berger A (2023) What is Log4Shell? The Log4j vulnerability explained (and what to do about it) — dynatrace.com. `https://www.dynatrace.com/news/blog/what-is-log4shell/`, log4j vulnerability explained

Bhattacharya P, Neamtiu I (2011) Assessing programming language impact on development and maintenance: A study on c and c++. In: Proceedings of the 33rd International Conference on Software Engineering, Association for

Computing Machinery, New York, NY, USA, ICSE '11, p 171–180, DOI 10.1145/1985793.1985817, URL https://doi.org/10.1145/1985793.1985817

Bi T, Xia B, Xing Z, Lu Q, Zhu L (2023) On the way to sboms: Investigating design issues and solutions in practice. arXiv preprint arXiv:230413261

Borges H, Valente MT (2018) What's in a github star? understanding repository starring practices in a social coding platform. Journal of Systems and Software 146:112–129

Brudo B (2023) What Happens When an AI Company Falls Victim to a Software Supply Chain Vulnerability. https://scribesecurity.com, [Victim to a Software Supply Chain Vulnerability]

Brudo B (2024) Spdx vs. cyclonedx. https://scribesecurity.com/blog/spdx-vs-cyclonedx-sbom-formats-compared/, [What are SBOM Formats]

C P L Foundation (Accessed May 27, 2024) Community health analytics open source software. https://chaoss.community/kb-metrics-and-metrics-models/

Chopra A, Mo M, Dodson S, Beschastnikh I, Fels SS, Yoon D (2021) "@ alex, this fixes# 9": Analysis of referencing patterns in pull request discussions. Proceedings of the ACM on human-computer interaction 5(CSCW2):1–25

Cohen J (1992) A power primer. Tutorials in Quantitative Methods for Psychology 112, DOI 10.1037/0033-2909.112.1.155

Cox R (2019) Surviving software dependencies. ACM New York, NY, USA, vol 62, pp 36–43

Cramér H (1999) Mathematical methods of statistics, vol 43. Princeton university press

Das R (2023) Sbom formats compared. https://www.techtarget.com/searchsecurity/tip/SBOM-formats-compared-CycloneDX-vs-SPDX-vs-SWID-Tags, [CycloneDX vs. SPDX vs. SWID Tags]

Foundjem A, Eghan E, Adams B (2021) Onboarding vs. diversity, productivity and quality—empirical study of the openstack ecosystem. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, pp 1033–1045

Gisev N, Bell JS, Chen TF (2013) Interrater agreement and interrater reliability: key concepts, approaches, and applications. Research in Social and Administrative Pharmacy 9(3):330–338

Gkortzis A, Feitosa D, Spinellis D (2021) Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. Elsevier, vol 172, p 110653

Hendrick S (2022) Software bill of materials (sbom)and cybersecurity readiness. The Linux Foundation

ISO/IEC (2022) Swid. https://wwwisoorg/standard/45170html ISO/IEC JTC 1/SC 27

Jarczyk O, Gruszka B, Jaroszewicz S, Bukowski L, Wierzbicki A (2014) Github projects. quality analysis of open-source software. pp 80–94, DOI 10.1007/978-3-319-13734-6_6

Jiang J, Lo D, He J, Xia X, Kochhar PS, Zhang L (2017) Why and how developers fork what from whom in github. Empirical Software Engineering 22, DOI 10.1007/s10664-016-9436-6

Jiao J, Tseng MM, Ma Q, Zou Y (2000) Generic bill-of-materials-and-operations for high-variety production management. Concurrent Engineering 8(4):297–321, DOI 10.1177/1063293X0000800404

Kampenes VB, Dybå T, Hannay JE, Sjøberg DI (2007) A systematic review of effect size in software engineering experiments. Information and Software Technology 49(11-12):1073–1086

Kikas R, Dumas M, Pfahl D (2015) Issue dynamics in github projects. In: Product-Focused Software Process Improvement: 16th International Conference, PROFES 2015, Bolzano, Italy, December 2-4, 2015, Proceedings 16, Springer, pp 295–310

Ladisa P, Plate H, Martinez M, Barais O (2022) Taxonomy of attacks on open-source software supply chains

Linux (2023) Spdx. https://spdxorg/

Martin RA (2020) Visibility & control: Addressing supply chain challenges to trustworthy software-enabled things. In: 2020 IEEE Systems Security Symposium (SSS), pp 1–4, DOI 10.1109/SSS47320.2020.9174365

Mirakhorli M, Garcia D, Dillon S, Laporte K, Morrison M, Lu H, Koscinski V, Enoch C (2024) A landscape study of open source and proprietary tools for software bill of materials (sbom). arXiv preprint arXiv:240211151

Muirí ÉÓ (2019) Framing software component transparency: Establishing a common software bill of material (sbom). vol 12

Nocera S, Romano S, Di Penta M, Francese R, Scanniello G (2023) Software bill of materials adoption: A mining study from github. In: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 39–49

NTIA (2021a) Ntia software component transparency. `https://www.ntia.gov/other-publication/ntia-software-component-transparency`, [SBOM Virtual Multistakeholder Meeting]

NTIA (2021b) Survey of existing sbom formats and standards. `https://ntia.gov/sites/default/files/publications/sbom_formats_survey-version-2021_0.pdf`, [SBOM Formats and Standards]

OWASP (2021) Cyclonedx. https://cyclonedxorg/

OWASP (2023a) Official cyclonedx specification documentation json reference v1.4. `https://cyclonedx.org/docs/1.4/json/`, [CycloneDx Documentation]

OWASP (2023b) Official cyclonedx tool center. `https://cyclonedx.org/tool-center/`, [CycloneDx Tools]

Ray B, Posnett D, Devanbu P, Filkov V (2017) A large-scale study of programming languages and code quality in github. Commun ACM 60(10):91–100, DOI 10.1145/3126905, URL `https://doi.org/10.1145/3126905`

Remaley EL, Remaley D (2021) Ntia sbom effort is progressing well. NTIA

Sanfilippo S (2023) Redis — redis.io. `https://redis.io/`, [Redis Library]

SPDX (2023a) Official SPDX Open Source Tools. `https://spdx.dev/tools-community/`, [Open Source SPDX Tools]

SPDX (2023b) Official SPDX Proprietary Tools. `https://spdx.dev/tools-commercial/`, [Commercial Proprietary SPDX Tools]

SPDX (2023c) Official SPDX Specification Documentation v2.3. `https://spdx.github.io/spdx-spec/v2.3/package-information/`, [SPDX Documentation]

van der Storm T (2007) Component-based configuration, integration and delivery

Storm VD, Tijs, et al. (2007) Component-based configuration, integration and delivery

Telecommunications N, (NTIA) IA (2021a) Minimum Elements for a Software Bill of Materials (SBOM). URL `https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf`, [NTIA2021]

Telecommunications N, (NTIA) IA (2021b) Sbom tool classification taxonomy. URL `https://www.ntia.gov/files/ntia/publications/ntia_sbom_tooling_taxonomy-2021mar30.pdf`, [NTIATaxonomy]

Tomczak M, Tomczak E (2014) The need to report effect size estimates revisited. an overview of some recommended measures of effect size. Trends in Sport Sciences

Waltermire DA, Feldman L, Witte GA (2016) Improving security and software management through the use of swid tags. David A. Waltermire, Larry Feldman, Gregory A. Witte

Wan Z, Xia X, Lo D, Murphy GC (2019) How does machine learning change software development practices? IEEE Transactions on Software Engineering 47(9):1857–1871

Wilcoxon F, Katti S, Wilcox RA, et al. (1970) Critical values and probability levels for the wilcoxon rank sum test and the wilcoxon signed rank test. Selected tables in mathematical statistics 1:171–259

Worthington J (2023) The world lags with sbom requirements, but likely not for long. Forrester

Xia B, Bi T, Xing Z, Lu Q, Zhu L (2023a) An empirical study on software bill of materials: Where we stand and the road ahead. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 2630–2642

Xia B, Zhang D, Liu Y, Lu Q, Xing Z, Zhu L (2023b) Trust in software supply chains: Blockchain-enabled sbom and the aibom future. arXiv preprint arXiv:230702088

Zerouali A, Mens T, Robles G, Gonzalez-Barahona JM (2019) On the diversity of software package popularity metrics: An empirical study of npm. In: 2019 IEEE 26th international conference on software analysis, Evolution and Reengineering (SANER), IEEE, pp 589–593

A Appendix

Table 10: Manual classification of 170 SBOM tools in NTIA's tool taxonomy.

			NTIA Tool Taxonomy Usecase								
			Produce			**Consume**			**Transform**		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
action-owasp-dependecy-track-check	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
apt2sbom	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Auditjs	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
Augur	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
bom	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Cavil	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗
cdxgen	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Checkov	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
CodeNotary CAS	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
Codenotary CAS Authenticate Docker Image and SBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Codenotary CAS Notarize Docker Image and SBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Cosign	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Covenant	✓	✓	✓	✗	✗	✓	✗	✗	✓	✗	✓
cve-bin-tool	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
CycloneDX .NET Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX CLI	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✗
CycloneDX Core for Java	✓	✗	✓	✓	✗	✗	✗	✓	✗	✗	✓
CycloneDX for .NET	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
CycloneDX for Erlang/Elixir (Mix)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Erlang/Elixir (Rebar3)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Go	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Gradle	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Maven	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Node.js	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for NPM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for SBT (Scala)	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX for Webpack	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX JavaScript Library	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Libraries for .NET	✓	✗	✓	✗	✗	✗	✗	✓	✗	✗	✓
CycloneDX library for Go	✓	✗	✓	✗	✗	✗	✗	✓	✗	✗	✗
CycloneDX PHP Composer Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX PHP Library	✓	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗
CycloneDX Python Generate SBOM	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Python Library	✓	✗	✓	✗	✗	✓	✓	✓	✓	✗	✓
CycloneDX Rust	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
CycloneDX Web Tool	✓	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗
cyclonedx-merge	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗
cyclonedx-npm-pipe	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
DaggerBoard	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Dependency-Track Maven Plugin	✓	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
Distro2SBOM	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
docker-sbom-cli-plugin	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
EMBA	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
FOSSLight	✓	✗	✗	✓	✓	✓	✗	✗	✓	✓	✗
FOSSology	✓	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓
Generate SBoM for Elixir project	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
gh-sbom	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
GitHub Self-Service SBOMs	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
gobom	✓	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗
Grype	✓	✓	✓	✗	✗	✓	✗	✓	✗	✗	✗
in-toto	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Jake	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗
jbom	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
kbom - Kubernetes Bill of Materials powered by KSOC	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
KubeClarity	✓	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓
Kyverno	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Lagoon Insights Handler	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✓
Lib4sbom	✓	✓	✓	✗	✓	✗	✗	✓	✗	✗	✗
LicenseComplianceTool	✓	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗
macaron	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓
mdbom	✓	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗
MLBOMdoc	✓	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Nexus Lifecycle Jenkins Plugin	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Nix / Nixpkgs	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
ntia-conformance-checker	✓	✗	✗	✓	✗	✗	✗	✓	✗	✗	✓
OSS Review Toolkit (ORT)	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗
OSS Inventory	✓	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Parlay	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗
Protobom	✓	✓	✓	✗	✗	✗	✗	✓	✓	✗	✗
REuse	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Salus	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
SBOM Assembler	✓	✓	✗	✗	✗	✗	✗	✓	✗	✓	✗
SBOM CLI	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
SBOM Explorer	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗
SBOM Grep	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
SBOM-Manager	✓	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
sbom-submission-action	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓
sbom-swissarmy-bitbucket-pipe	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
sbom-tool	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
SBOM4Files	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
SBOMAudit	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
sbomqs	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
SBOMTrend	✓	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗
sca-codeinsight-reports-cyclonedx	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Scancode Toolkit	✓	✓	✗	✓	✗	✓	✗	✗	✗	✗	✗
SecObserve	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Semgrep	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
SnykVulnCheck	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
SPDX Golang Libraries	✓	✗	✓	✓	✗	✓	✗	✓	✓	✗	✓
SPDX Java Libraries and Tools	✓	✗	✓	✓	✗	✓	✓	✓	✓	✓	✓
SPDX JavaScript Libraries	✓	✗	✗	✓	✗	✗	✓	✗	✓	✗	✓
SPDX Maven Plugin	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
SPDX Online Tools	✓	✗	✗	✓	✓	✓	✓	✓	✓	✗	✓
SPDX Python Libraries	✓	✗	✓	✓	✗	✗	✗	✓	✓	✗	✓
spdx-sbom-generator	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
spdxcyclone	✓	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗
SRC:CLR SBOM Generator	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
SW360	✓	✗	✗	✗	✗	✓	✓	✓	✗	✓	✗
SwiftBOM	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
Syft	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗	✓
Tern	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
ThreatMapper	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Valaa Stack	✓	✗	✓	✗	✗	✗	✗	✗	✓	✗	✗
Vexy	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
vsm-sbom-booster	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
Vuls	✓	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
yasca (Yet Another SCA tool)	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗
Yocto Project / OpenEmbedded	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Apiiro	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Arnica	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Arsenal	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✗
Athena	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Beniva Software Bill of Materials (SBOM)	✗	✗	✗	✓	✗	✓	✗	✓	✗	✗	✗
Black Duck	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
BlackBerry Jarvis	✗	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
Bytesafe	✗	✓	✗	✓	✗	✓	✗	✓	✗	✗	✓
CAST Highlight	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
Codenotary vcn	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
CodeSentry	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Contrast Security	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
CxSCA	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
Cybeats SBOM Studio	✗	✓	✓	✓	✗	✗	✗	✓	✓	✓	✓
Cybellum SBOM	✗	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓
Debricked	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓
DejaCode	✗	✗	✓	✓	✓	✓	✗	✗	✓	✗	✗
Endor Labs	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Enso	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
FACT	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Flawnter	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓
Fortify on Demand	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Fortress File Integrity Assurance	✗	✓	✓	✓	✗	✓	✓	✓	✓	✗	✓
FOSSA	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
FOSSID	✗	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗
Heimdall	✗	✓	✓	✓	✗	✓	✗	✓	✓	✗	✗
Interlynk	✗	✓	✓	✓	✗	✓	✗	✓	✓	✓	✗
Ion Channel Platform	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
JDisc Discovery	✗	✗	✓	✓	✗	✓	✗	✓	✗	✗	✓
JupiterOne	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Kondukto	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
Manifest	✗	✓	✓	✓	✗	✓	✓	✓	✗	✓	✓
MedScan	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
Mend SCA	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
NetRise Turbine	✗	✓	✓	✓	✗	✓	✗	✓	✗	✓	✗
Nexus IQ	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
NowSecure Platform	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
ONEKEY firmware analysis platform	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
Prisma Cloud	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
PulseUno Plugin for Dimensions CM	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✗
Reliza Hub	✗	✗	✓	✗	✗	✓	✗	✓	✗	✗	✓
Rezilion Dynamic SBOM	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
RKVST SBOM Hub	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗
SBOM Benchmark	✗	✓	✗	✓	✗	✓	✓	✗	✗	✗	✗
SBOM Insights	✗	✓	✓	✓	✓	✓	✗	✓	✗	✓	✓
SBOM Observer	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
SBOM.sh	✗	✓	✓	✗	✗	✓	✗	✓	✗	✗	✗
SCANOSS	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓
SecureStack	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓
Snyk	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
Software Assurance Guardian Point Man	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
Sonatype Lift	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
SourceAuditor	✗	✗	✗	✓	✗	✓	✓	✓	✓	✗	✗
StackAware	✗	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗
Tidelift	✗	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
TrustSource	✗	✓	✗	✓	✓	✓	✗	✓	✗	✗	✓

Continued on next page

Table 10 – continued from previous page

			NTIA Tool Categories								
			Produce			Consume			Transform		
Name	Opensource	Supports Both Formats	Build	Analyze	Edit	View	Diff	Import	Translate	Merge	Support
Veracode	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Vigilant Ops	✗	✓	✓	✓	✗	✓	✗	✗	✓	✗	✓
Vulert	✗	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
Vulnerabilities.io	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
Xray	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗
Xygeni Software Supply-Chain Security	✗	✓	✓	✗	✗	✓	✗	✗	✗	✗	✓